Predicting NBA Fouls: Scouting the Referees

Technical Report — Multiple Linear Regression

Jackson Ferry, Jake Hough, Whitney Holt

2026-10-17

Table of contents

1 Plan 2
1.1 Business Objective 2
1.2 Actions and States of the World 2
1.3 Loss Function 2
1.4 Ideal Data and the Data Story 2
2 Build 3
2.1 Model Specification 3
2.2 Simulating Ideal Data and Recovering Parameters 3
3 Explore 4
3.1 Data Sources 4
3.2 Comparing Real Data to Ideal Data 5
3.3 Data Cleaning and Merging 5
3.3.1 Combining the Referee Stats Files Into One Table 5
3.3.2 Combining CHIEF and CREW Rows 6
3.3.3 Cleaning the Games File 6
3.3.4 Matching Referee Names and Building Crew Statistics 6
3.3.5 Pulling Team Game Logs 7
3.3.6 Total Fouls Per Game (Outcome) 8
3.3.7 Team Foul Averages (Home and Away) 9
3.3.8 Final Modeling Table 9
3.4 Exploratory Data Analysis 10
3.4.1 Distribution of Total Fouls 10
3.4.2 Foul Levels Change Each Season 12
3.4.3 Does the Crew Matter? 13
3.4.4 Referees Are Consistent Year to Year 15
3.4.5 Some Referees Call Far More Fouls Than Others 16
3.4.6 Overtime Adds Fouls 17
3.4.7 Correlations 18
3.5 Data Limitations 19
4 Reconcile 19
4.1 Validity: Simulated vs. Real Outcomes 19
4.2 Feature Engineering 20
4.3 Fitting the Selected Model 20
4.4 Representativeness: Influential Games 21
4.5 Linearity 21
4.6 Independence 22
4.7 Constant Variance 23
4.8 Normality 23
4.9 Identifiability 25
5 Evaluate 26
5.1 Models Compared 26
5.2 Frequentist Results (OLS) 27
5.3 Bayesian Results 27
5.4 Overall Model Fit 29

5.5 Interpreting the Parameter Estimates 29
5.6 Visualizing the Interval Estimates 30
5.7 Predictive Fit on Test Data 32
5.8 Decision-Theoretic Fit: Expected Quadratic Loss 33
6 Recommendations 34
6.1 Translating Crew Differences Into Free-Throw Points 34
6.2 Case Study: Lakers vs. Warriors, October 13, 2026 34
6.3 Recommendations 36
6.4 Limitations and Next Steps 36

1 Plan

1.1 Business Objective

NBA teams spend significant time scouting opponents, but rarely scout the officials. Referee crews differ in how tightly they call games: some crews call many fouls, while others "let the players play." The number of fouls in a game affects free-throw points, foul trouble for key players, and how physical a team can afford to be on defense.

The decision we want to inform is a **pre-game coaching decision**. The referee crew for every NBA game is published at 9:00 AM ET on game day. Using that crew, a coaching staff must decide how many fouls to plan for and adjust its game plan accordingly (rotations for foul-prone players and how physical to play on defense).

Our objective is therefore to **predict the total number of personal fouls called in a regular-season game before tip-off**, and to quantify how much of that total is driven by the referee crew compared with the two teams.

1.2 Actions and States of the World

- **Actions (a):** the number of fouls the coaching staff plans for, and the matching game plan. In practice this is grouped into three plans: *physical defense* (low-foul crew expected), *standard plan*, and *protect foul-prone players / safer defense* (high-foul crew expected).
- **States of the world (θ):** the true number of fouls that will be called in the game, which is unknown before tip-off.

1.3 Loss Function

We formalize the objective with **quadratic (squared error) loss**:

$$\ell(a, \theta) = (a - \theta)^2$$

where a is the planned (predicted) number of total fouls and θ is the actual number of total fouls. We chose quadratic loss over linear utility for two reasons:

1. **Large misses hurt more than small misses.** Under-planning by 2 fouls is a minor inconvenience; under-planning by 10 fouls can put a starter on the bench with foul trouble in the third quarter. The cost of a miss grows faster than linearly.
2. **Misses in both directions are costly.** Planning for too many fouls makes a team play too passively; planning for too few leads to foul trouble. Quadratic loss is symmetric.

Under quadratic loss, the action that minimizes expected loss is the **mean** of the predictive distribution, $a^* = E[\theta \mid X]$. This aligns directly with ordinary least squares, which estimates the conditional mean by minimizing squared error. The mean squared error (MSE) on held-out games is therefore our decision-theoretic measure of fit.

To translate fouls into a business quantity, we also convert fouls into **free-throw points**. *Assumption:* each foul produces about 1.1 free-throw attempts on average, and NBA teams make about 78% of free throws, so

$$\text{free-throw points} \approx 0.85 \times \text{fouls} \ .$$

This conversion is approximate and is used only to communicate the size of the crew effect, not to estimate the model.

1.4 Ideal Data and the Data Story

Data story. Before a game, the league assigns a crew of three referees. Each referee has a personal tendency to call more or fewer fouls. The two teams also have their own tendencies to commit fouls (style of play, roster, pace). The game context matters too: overtime adds playing time, and the end of the season may change effort and rotations. League-wide rule emphasis changes from season to season. Total fouls in the game are the result of all of these factors plus random game-to-game variation.

Ideal data. One row per game with:

- **Outcome (Y):** total personal fouls called in the game (both teams combined).
- **Predictors (X):** each referee's foul tendency measured *before* the game, the crew's experience and any home/road bias, each team's foul tendency measured before the game, overtime periods, timing in the season, and the season.

2 Build

2.1 Model Specification

Because we expect the factors above to add together, and because total fouls are continuous and roughly bell-shaped (shown in the Explore section), we start with a multiple linear regression:

$$\begin{aligned}\text{total_fouls}_i = &\beta_0 + \beta_1\,\text{crew_fouls}_i + \beta_2\,\text{crew_experience}_i + \beta_3\,\text{crew_foul_diff}_i + \beta_4\,\text{crew_home_win_pct}_i \\ &+\beta_5\,\text{home_team_fouls}_i + \beta_6\,\text{away_team_fouls}_i + \beta_7\,\text{overtime}_i + \beta_8\,\text{late_season}_i \\ &+\sum_s \gamma_s\,\text{season}_{s,i} + \epsilon_i, \qquad \epsilon_i \sim \text{Normal}(0, \sigma^2)\end{aligned}$$

Simplifying assumptions:

- **Crew averaging.** The three referees' tendencies are averaged into one crew value. This assumes each referee contributes equally to the crew's total, regardless of role (crew chief vs. crew member).
- **Linear, additive effects.** Each factor adds a fixed number of fouls, regardless of the other factors (no interactions).
- **Normal errors.** Fouls are counts, so a Poisson model would be the natural choice for count data. Because total fouls are large counts (around 40) and approximately bell-shaped, the normal approximation is reasonable, and we stay within ordinary linear regression.

2.2 Simulating Ideal Data and Recovering Parameters

Before using real data, we simulate data from the model above with known ("true") parameter values, then check that OLS recovers them. The parameter values are chosen to look like plausible NBA games; they are not estimates.

```
import numpy as np
import polars as pl
import pandas as pd
import statsmodels.api as sm
from pyprojroot.here import here
from IPython.display import display
from plotnine import *

## Presentation colors - used on every chart
nbaGreen = '#3AA845'
nbaTheme = (theme_minimal()
  + theme(text = element_text(size = 11),
          plot_title = element_text(size = 13, weight = 'bold'),
          figure_size = (8, 4.5))
)

## To fix format of PDF output
pl.Config.set_tbl_cols(8);
pl.Config.set_tbl_rows(10);
```

```
## Simulate ideal data - one row per game
rng = np.random.default_rng(42)
nGames = 1000

## True parameter values (chosen, not estimated)
trueParams = {
    'const': -60.0,
    'crew_fouls_per_game': 1.5,   ## crew tendency matters most
    'home_team_fouls': 1.0,
    'away_team_fouls': 1.0,
    'overtime': 5.0,              ## extra 5 minutes of play
    'late_season': -1.5,          ## lighter effort / rotations late
}
sigma = 6.0                       ## game-to-game randomness

simData = pl.DataFrame({
```

```
    'crew_fouls_per_game': rng.normal(39.5, 1.7, nGames),
    'home_team_fouls': rng.normal(19.7, 1.4, nGames),
    'away_team_fouls': rng.normal(19.7, 1.4, nGames),
    'overtime': rng.binomial(1, 0.06, nGames),
    'late_season': rng.binomial(1, 0.10, nGames),
})

## Build the outcome from the model
mu = trueParams['const'] + sum(trueParams[c] * simData[c].to_numpy() for c in simData.columns)
simData = simData.with_columns(total_fouls = pl.Series(mu + rng.normal(0, sigma, nGames)).round(0))

## Recover parameters with OLS
X_sim = sm.add_constant(simData.drop('total_fouls').to_pandas())
fit_sim = sm.OLS(simData['total_fouls'].to_pandas(), X_sim).fit()

ciSim = fit_sim.conf_int()
pl.DataFrame({
    'parameter': list(trueParams.keys()),
    'true': list(trueParams.values()),
    'estimate': fit_sim.params[list(trueParams.keys())].round(2).tolist(),
    'ci_low': ciSim.loc[list(trueParams.keys()), 0].round(2).tolist(),
    'ci_high': ciSim.loc[list(trueParams.keys()), 1].round(2).tolist(),
})
```

parameter	true	estimate	ci_low	ci_high
str	f64	f64	f64	f64
"const"	−60.0	−59.79	−71.37	−48.21
"crew_fouls_per_game"	1.5	1.57	1.35	1.8
"home_team_fouls"	1.0	1.05	0.78	1.31
"away_team_fouls"	1.0	0.81	0.54	1.08
"overtime"	5.0	5.68	4.07	7.29
"late_season"	−1.5	−1.6	−3.01	−0.19

Every slope estimate is close to its true value, and every 95% confidence interval for a slope contains the true value. The intercept is less precise because it represents a game where every predictor equals zero, far outside the data, but it is not of business interest. This confirms that, if real games follow our data story, OLS can recover the effects we care about, and our code works before we use real data.

3 Explore

3.1 Data Sources

We combined three sources into one table with one row per game:

Source	Level	What it provides	Link
NBAstuffer referee stats (2017–18 to 2025–26)	One row per referee per season	Fouls called per game, experience, foul differential, home-team win %	https://www.nbastuffer.com/
NBA game referee assignments	One row per game	Teams, date, season, and the three officials	**[ADD LINK]**
NBA team game logs (`nba_api`, `LeagueGameLog`)	One row per team per game	Personal fouls (`PF`) and minutes played	https://github.com/swar/nba_api

3.2 Comparing Real Data to Ideal Data

Our real data matches the ideal data closely on the outcome: total fouls are measured exactly for every game. The predictors differ from the ideal in three ways:

1. **Referee tendency is a season average, not a pre-game measure.** NBAstuffer reports each referee's average for the full season, which includes the game being predicted. Ideally we would use only games before tip-off. This is a form of data leakage and likely makes the crew effect look somewhat stronger in-sample than it would be in true pre-game use. We show in the EDA that referees are consistent year to year, which supports using last season's averages for real pre-game predictions (as we do in the Lakers–Warriors example).
2. **Team foul tendency is also a season average** computed from the game logs, with the same limitation.
3. **Some ideal predictors are not available:** player-level fouls and minutes, foul types, and team strategy. Team foul averages act as a **proxy** for team style.

Also, `overtime` is not known before tip-off. It is included as a control so the other coefficients are not distorted by longer games; for pre-game predictions we set it to zero.

3.3 Data Cleaning and Merging

3.3.1 Combining the Referee Stats Files Into One Table

```
seasons = ['2017-2018', '2018-2019', '2019-2020', '2020-2021', '2021-2022', '2022-2023', '2023-2024', '2024-2025',
'2025-2026']

refList = []
for s in seasons:
    fileName = f'nba_referee_stats_{s}.csv'
    if s == '2025-2026':
        fileName = 'nba_referee_stats_2025_2026.csv' ## uses an underscore instead of dash between years

    refSeason = pl.read_csv(here(f'NBAFoulPredictor/Data/{fileName}'))

    ## find foul differential column (the name changes across seasons)
    diffCol = [c for c in refSeason.columns if 'FOUL DIFFERENTIAL' in c][0]

    refSeason = (refSeason
        .rename({diffCol: 'foul_diff'})
        .rename({'TYPE': 'ROLE'}, strict = False)           ## older files say TYPE
        .drop(['RANK', 'Unnamed: 13'], strict = False)      ## empty columns
        .with_columns(Season = pl.lit(s))                   ## adds Season if missing
        )
    refList.append(refSeason)

## Stack all seasons
refStats = pl.concat(refList, how = 'diagonal')

## Renaming columns
refStats = refStats.rename({
    'Season': 'season', 'REFEREE': 'referee', 'ROLE': 'role', 'GENDER': 'gender', 'EXPERIENCE (YEARS)': 'experience',
'GAMES OFFICIATED': 'games', 'HOME TEAM WIN%': 'home_win_pct', 'HOME TEAM POINTS DIFFERENTIAL': 'home_pt_diff',
'TOTAL POINTS PER GAME': 'points_per_game', 'CALLED FOULS PER GAME': 'fouls_per_game', 'FOUL% AGAINST ROAD TEAMS':
'foul_pct_road', 'FOUL% AGAINST HOME TEAMS': 'foul_pct_home',
})

## Some seasons store percents as 53.8 instead of 0.538
refStats = refStats.with_columns(
    pl.when(pl.col(c) > 1).then(pl.col(c) / 100).otherwise(pl.col(c)).alias(c)
    for c in ['foul_pct_road', 'foul_pct_home']
)
refStats.shape
```

```
(1101, 13)
```

3.3.2 Combining CHIEF and CREW Rows

Starting in 2020–21, a referee can appear twice in a season (once as crew chief, once as crew member). We combine these rows into one row per referee per season, using a games-weighted average.

```
avgCols = ['home_win_pct', 'home_pt_diff', 'points_per_game', 'fouls_per_game', 'foul_pct_road', 'foul_pct_home',
'foul_diff']

refStats = (refStats
    .group_by(['season', 'referee'])
    .agg(
        experience = pl.col('experience').max(),
        gender = pl.col('gender').first(),
        games = pl.col('games').sum(),
        ## games-weighted average for every column in avgCols
        *[((pl.col(c) * pl.col('games')).sum() / pl.col('games').sum()).alias(c) for c in avgCols]
    )
    .sort(['season', 'referee'])
)
refStats.shape
```

```
(696, 12)
```

3.3.3 Cleaning the Games File

```
games = pl.read_csv(here('NBAFoulPredictor/Data/nba_games_ref_assignments.csv'))

games = (games
    ## Split season code (22025 --> game type 2, start year 2025)
    .with_columns(
        game_type = pl.col('season') // 10000,
        start_year = pl.col('season') % 10000,
    )
    ## Match the referee table format ('2025-2026')
    .with_columns(
        season = pl.col('start_year').cast(pl.String) + '-' + (pl.col('start_year') + 1).cast(pl.String)
    )
    ## Keep only regular season games from 2017-18 on
    .filter(pl.col('game_type') == 2, pl.col('start_year') >= 2017)
    ## Fix team names
    .with_columns(
        pl.col(['home_team', 'away_team']).str.replace('L.A.', 'LA', literal = True)
    )
    ## Remove games listed twice
    .unique(subset = 'game_id', keep = 'first', maintain_order = True)
    ## Remove games missing referees
    .drop_nulls(subset = ['official1', 'official2', 'official3'])
    ## Drop 4th referee column (empty during regular season)
    .drop(['official4', 'official4_code', 'official4_JNum'])
)
games.shape
```

```
(10774, 21)
```

3.3.4 Matching Referee Names and Building Crew Statistics

Referee names are spelled differently across files, so we standardize them before joining. After the fix, every referee in every game matches a season record.

```
nameFix = {
    'J.T. Orr': 'JT Orr',
    'Sharae Mitchell': "Sha'Rae Mitchell", 'ShaRae Mitchell': "Sha'Rae Mitchell",
    'Pat OConnell': "Pat O'Connell", 'PAT OConnell': "Pat O'Connell", "PAT O'Connell": "Pat O'Connell",
    'Cheryl Flores': 'Che Flores',
    'Jamahl Ralls': 'JD Ralls',
    'CP Painter': 'Chelisa Painter',
}
refStats = refStats.with_columns(pl.col('referee').replace(nameFix))

## One row per referee per game, with that referee's season stats
gameRefs = (games
    .unpivot(index = ['game_id', 'season'], on = ['official1', 'official2', 'official3'],
        variable_name = 'slot', value_name = 'referee')
    .with_columns(pl.col('referee').replace(nameFix))
    .join(refStats, on = ['season', 'referee'], how = 'left')
)
## Check - should be 0 unmatched referees
gameRefs['fouls_per_game'].null_count()
```

```
0
```

```
## Average the 3 referees into one crew value per game
crewStats = (gameRefs
    .group_by('game_id')
    .agg(
        crew_fouls_per_game = pl.col('fouls_per_game').mean(),
        crew_experience = pl.col('experience').mean(),
        crew_foul_diff = pl.col('foul_diff').mean(),
        crew_home_win_pct = pl.col('home_win_pct').mean(),
    )
)
games = games.join(crewStats, on = 'game_id', how = 'left')
games.shape
```

```
(10774, 25)
```

3.3.5 Pulling Team Game Logs

The NBA API pull below was run once and saved to `Data/nba_team_game_logs.csv`, so it is commented out to avoid calling the API on every render.

```
# import time
# from nba_api.stats.endpoints import leaguegamelog
#
# foulList = []
# for year in range(2017, 2026):
#     seasonCode = f'{year}-{str(year + 1)[-2:]}' ## nba_api wants '2017-18' not '2017-2018'
#     log = leaguegamelog.LeagueGameLog(
#         season = seasonCode,
#         season_type_all_star = 'Regular Season',
#         timeout = 60,
#     )
#     result = log.get_dict()['resultSets'][0]
#     seasonLog = pl.DataFrame(result['rowSet'], schema = result['headers'], orient = 'row', infer_schema_length = None)
#     foulList.append(seasonLog.select('GAME_ID', 'TEAM_ABBREVIATION', 'PF', 'MIN'))
```

```
#     time.sleep(1) ## pause so the NBA site doesn't block us
#
# gameLogs = pl.concat(foulList)
# gameLogs.write_csv(here('Data/nba_team_game_logs.csv'))

gameLogs = pl.read_csv(here('NBAFoulPredictor/Data/nba_team_game_logs.csv'))
gameLogs['MIN'].value_counts().sort('MIN')  ## 240 = regulation, each +25 = one overtime
```

MIN	count
i64	u32
240	20372
265	994
290	120
315	10
340	2

3.3.6 Total Fouls Per Game (Outcome)

```
totalFouls = (gameLogs
    .with_columns(game_id = pl.col('GAME_ID').cast(pl.Int64)) ## 0021700001 --> 21700001
    .group_by('game_id')
    .agg(
        total_fouls = pl.col('PF').sum(),     ## both teams' fouls added together
        n_teams = pl.len(),                   ## should be 2 every game
        game_minutes = pl.col('MIN').max(),
    )
)
games = games.join(totalFouls, on = 'game_id', how = 'left')

## Checks: every game has exactly 2 teams, and how many games have no foul total
print(totalFouls['n_teams'].value_counts())
print('Games missing total_fouls:', games['total_fouls'].null_count())
games.filter(pl.col('total_fouls').is_null()).group_by('season').len()
```

```
shape: (1, 2)
┌──────────┬─────────┐
│ n_teams ┆ count │
│ ---     ┆ ---   │
│ u32     ┆ u32   │
╞══════════╪═════════╡
│ 2       ┆ 10749 │
└──────────┴─────────┘
Games missing total_fouls: 45
```

season	len
str	u32
"2019-2020"	45

All 45 games without a foul total are from the 2019–20 season, which was interrupted by COVID-19. They are 0.4% of the data, so we drop them.

```
games = (games
    .drop_nulls(subset = 'total_fouls')
    .drop('n_teams')
)
games.shape
```

```
(10729, 27)
```

3.3.7 Team Foul Averages (Home and Away)

We first tried the NBAstuffer team fouls file, but it stops at 2024–25, which left every 2025–26 game blank. Instead, we calculated each team's season foul average ourselves from the game logs, which cover every season.

```
teamSeasonFouls = (gameLogs
    ## Get season from game ID
    .with_columns(start_year = 2000 + (pl.col('GAME_ID') // 100000) % 100)
    .with_columns(
        season = pl.col('start_year').cast(pl.String) + '-' + (pl.col('start_year') + 1).cast(pl.String)
    )
    ## Average fouls per game for each team in each season
    .group_by(['season', 'TEAM_ABBREVIATION'])
    .agg(team_fouls = pl.col('PF').mean())
)

games = (games
       .join(teamSeasonFouls.rename({'TEAM_ABBREVIATION': 'home_team_abbr', 'team_fouls': 'home_team_fouls'}), on =
['season', 'home_team_abbr'], how = 'left')
       .join(teamSeasonFouls.rename({'TEAM_ABBREVIATION': 'away_team_abbr', 'team_fouls': 'away_team_fouls'}), on =
['season', 'away_team_abbr'], how = 'left')
)
games.select(pl.col('home_team_fouls', 'away_team_fouls').null_count())
```

home_team_fouls	away_team_fouls
u32	u32
0	0

3.3.8 Final Modeling Table

We create `overtime` (number of overtime periods) and `late_season` (game in the last 14 days of the regular season), keep only the useful columns, and save the cleaned table.

```
gamesClean = (games
    .with_columns(game_date = pl.col('game_date').str.to_date('%m/%d/%Y'))
    .with_columns(
        overtime = ((pl.col('game_minutes') - 240) / 25).cast(pl.Int64),
        days_left = (pl.col('game_date').max().over('season') - pl.col('game_date')).dt.total_days(),
    )
    .with_columns(late_season = pl.col('days_left') <= 14)
    .select(
        'game_id', 'game_date', 'season', 'home_team_abbr', 'away_team_abbr', 'official1', 'official2', 'official3',
        'crew_fouls_per_game', 'crew_experience', 'crew_foul_diff', 'crew_home_win_pct',
        'home_team_fouls', 'away_team_fouls', 'overtime', 'late_season', 'total_fouls'
    )
    .sort('game_date')
)
gamesClean.write_parquet(here('NBAFoulPredictor/Data/nba_games_clean.parquet'))
gamesClean.shape
```

```
(10729, 17)
```

The final table has **10,729 games** across **9 regular seasons** (2017–18 through 2025–26), with one row per game.

Data dictionary:

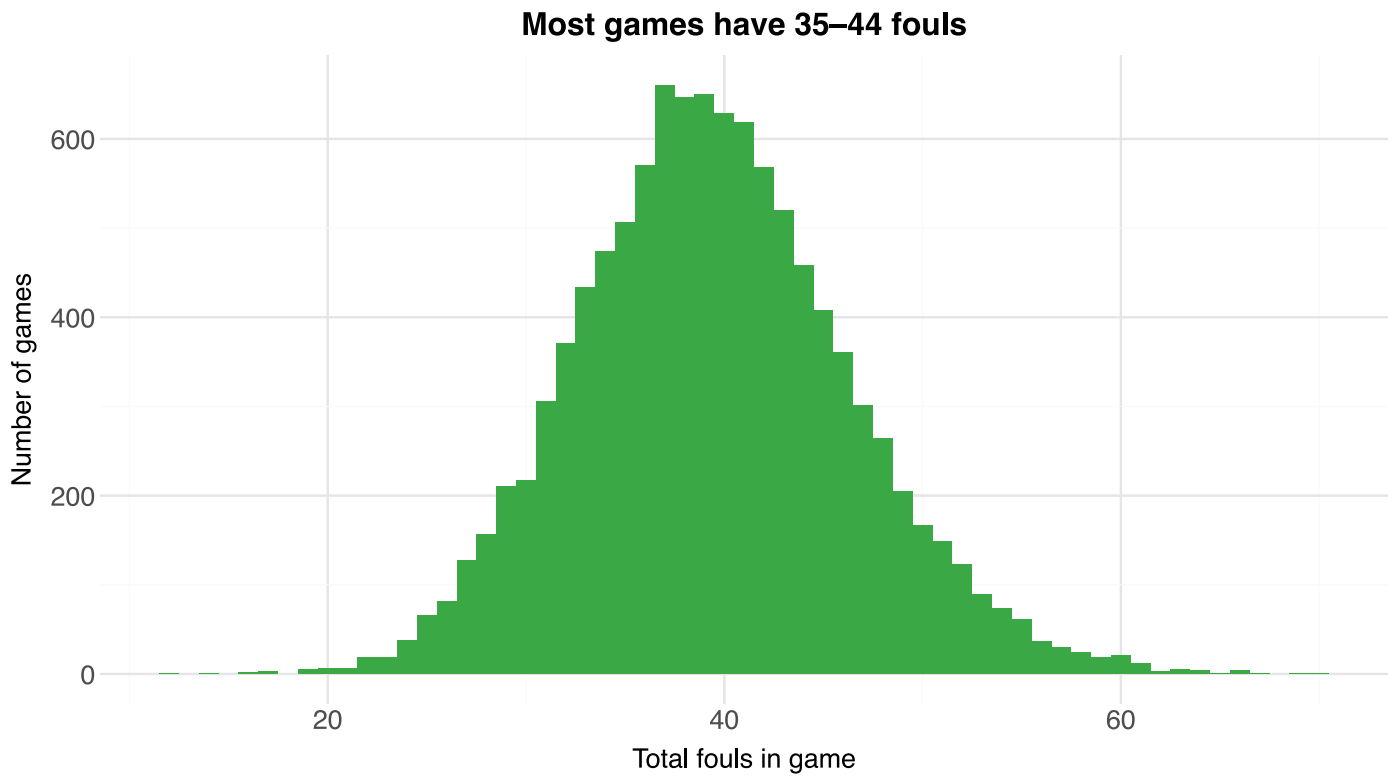
Variable	Description
`total_fouls`	**Outcome.** Personal fouls called on both teams combined
`crew_fouls_per_game`	Average of the three referees' season fouls called per game
`crew_experience`	Average years of NBA experience of the three referees
`crew_foul_diff`	Average of (fouls on road team − fouls on home team) per game
`crew_home_win_pct`	Average home-team win % in the referees' games
`home_team_fouls`, `away_team_fouls`	Each team's average personal fouls per game that season
`overtime`	Number of overtime periods (0–4)
`late_season`	Game in the final 14 days of the regular season
`season`	Season (2017–18 is the reference level)

Note on sample size: the assignment suggests at most 5,000 rows to reduce computation time. All models in this report fit in seconds on the full 10,729 games, so we use the full data for more precise estimates.

3.4 Exploratory Data Analysis

3.4.1 Distribution of Total Fouls

```
(ggplot(gamesClean, aes(x = 'total_fouls'))
    + geom_histogram(binwidth = 1, fill = nbaGreen)
    + labs(x = 'Total fouls in game', y = 'Number of games',
           title = 'Most games have 35–44 fouls')
    + nbaTheme
)
```




```
gamesClean['total_fouls'].describe()
```

statistic	value
str	f64
"count"	10729.0
"null_count"	0.0
"mean"	39.452698
"std"	6.885991
"min"	12.0
"25%"	35.0
"50%"	39.0
"75%"	44.0
"max"	70.0

Total fouls are roughly bell-shaped and centered near 39.5 (median 39). The middle 50% of games have 35 to 44 fouls. The distribution has a slight right tail (a few games above 60 fouls), so we checked the extreme games below. None are data errors: the low-foul games are mostly late-season games, and the high-foul games include overtime games.

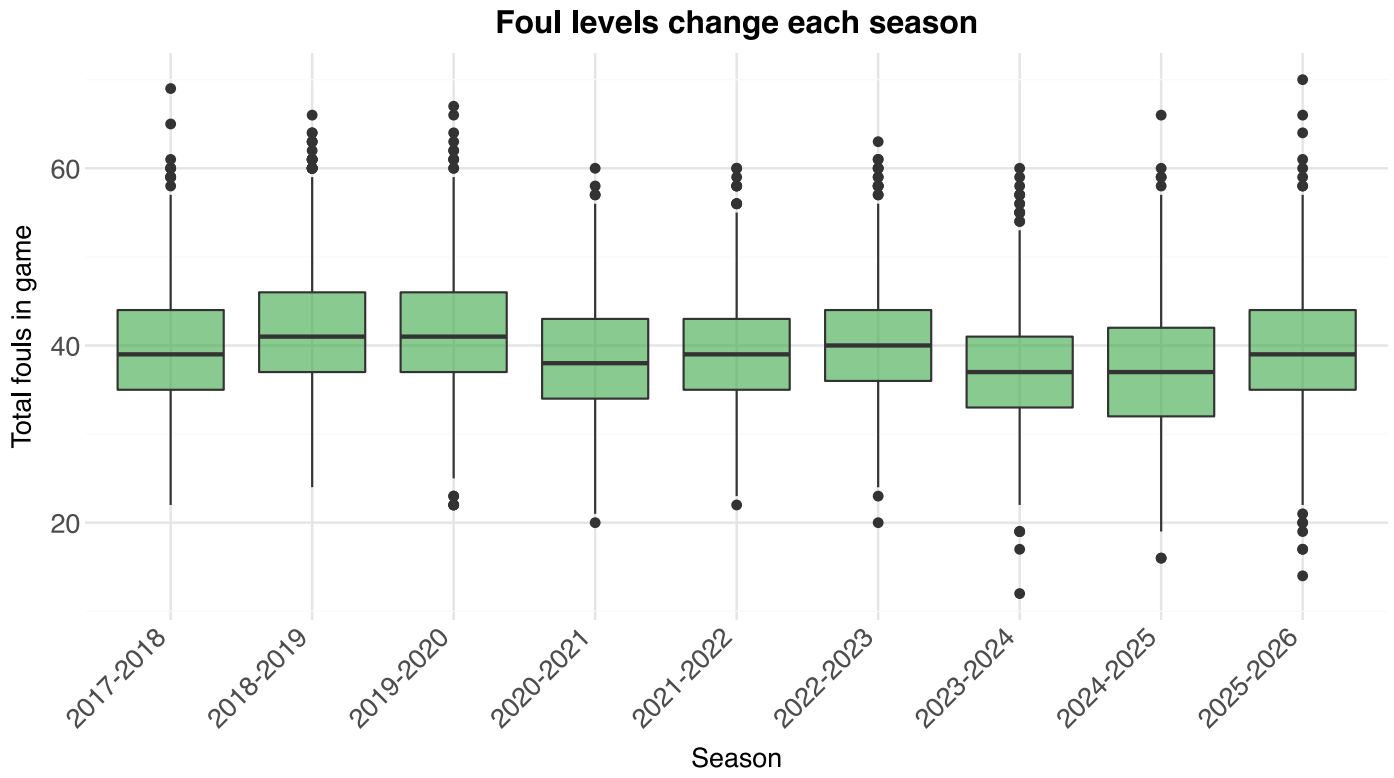
```
## Fewest and most fouls
lowGames = gamesClean.sort('total_fouls').select('game_date', 'home_team_abbr', 'away_team_abbr', 'overtime',
'late_season', 'total_fouls').head(5)
highGames = gamesClean.sort('total_fouls', descending = True).select('game_date', 'home_team_abbr', 'away_team_abbr',
'overtime', 'late_season', 'total_fouls').head(5)
display(lowGames)
display(highGames)
```

game_date	home_team_abbr	away_team_abbr	overtime	late_season	total_fouls
date	str	str	i64	bool	i64
2024-04-09	"MIL"	"BOS"	0	true	12
2026-01-12	"IND"	"BOS"	0	false	14
2025-04-11	"PHX"	"SAS"	0	true	16
2025-04-13	"SAC"	"PHX"	0	true	16
2024-03-02	"LAL"	"DEN"	0	false	17

game_date	home_team_abbr	away_team_abbr	overtime	late_season	total_fouls
date	str	str	i64	bool	i64
2025-10-23	"IND"	"OKC"	2	false	70
2017-11-29	"PHI"	"WAS"	0	false	69
2020-07-31	"DAL"	"HOU"	1	true	67
2018-11-09	"PHI"	"CHA"	1	false	66
2019-10-26	"PHX"	"LAC"	0	false	66

3.4.2 Foul Levels Change Each Season

```
(ggplot(gamesClean, aes(x = 'season', y = 'total_fouls'))
    + geom_boxplot(fill = nbaGreen, alpha = 0.6)
    + labs(x = 'Season', y = 'Total fouls in game',
           title = 'Foul levels change each season')
    + nbaTheme
    + theme(axis_text_x = element_text(rotation = 45, ha = 'right'))
)
```

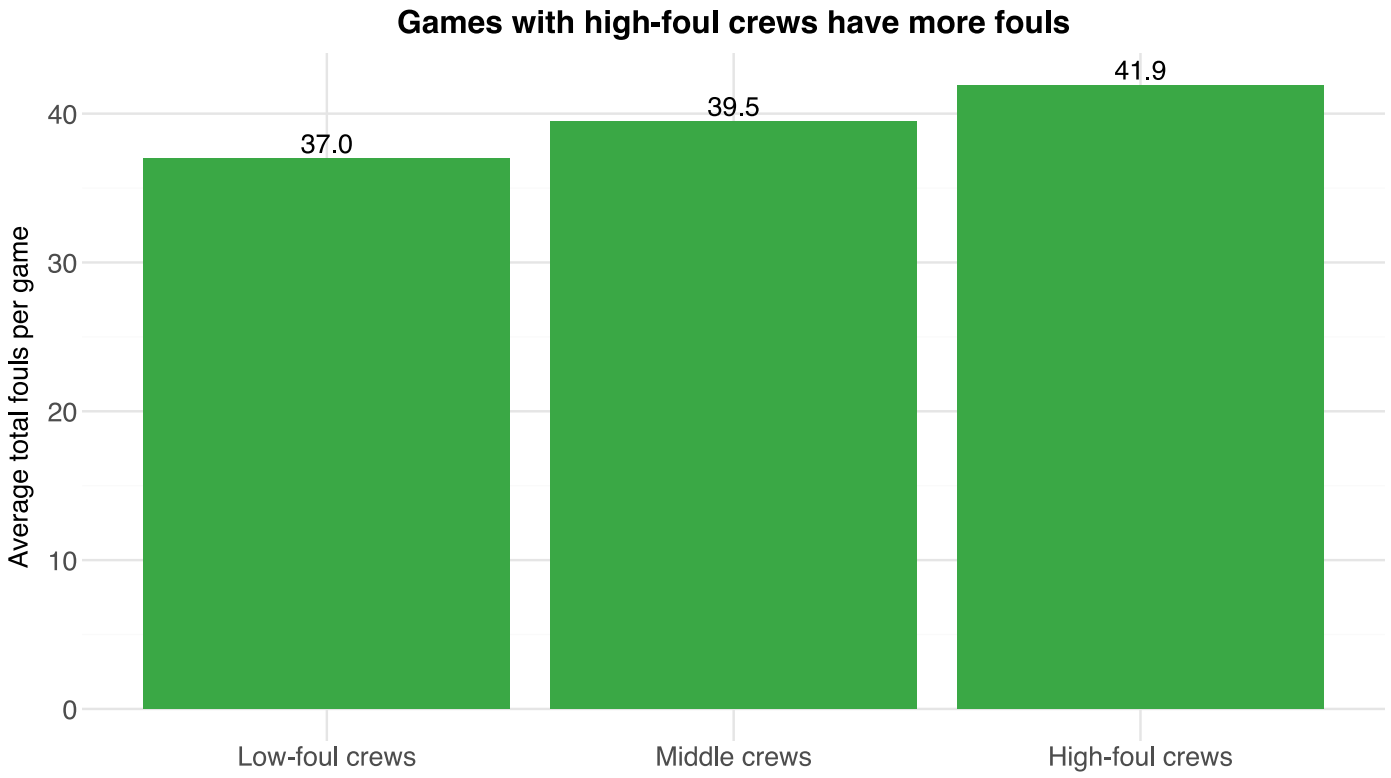


League-wide foul levels shift from season to season (for example, 2023–24 and 2024–25 are lower). This reflects rule emphasis and officiating guidance from the league, so we include season as a dummy-coded predictor.

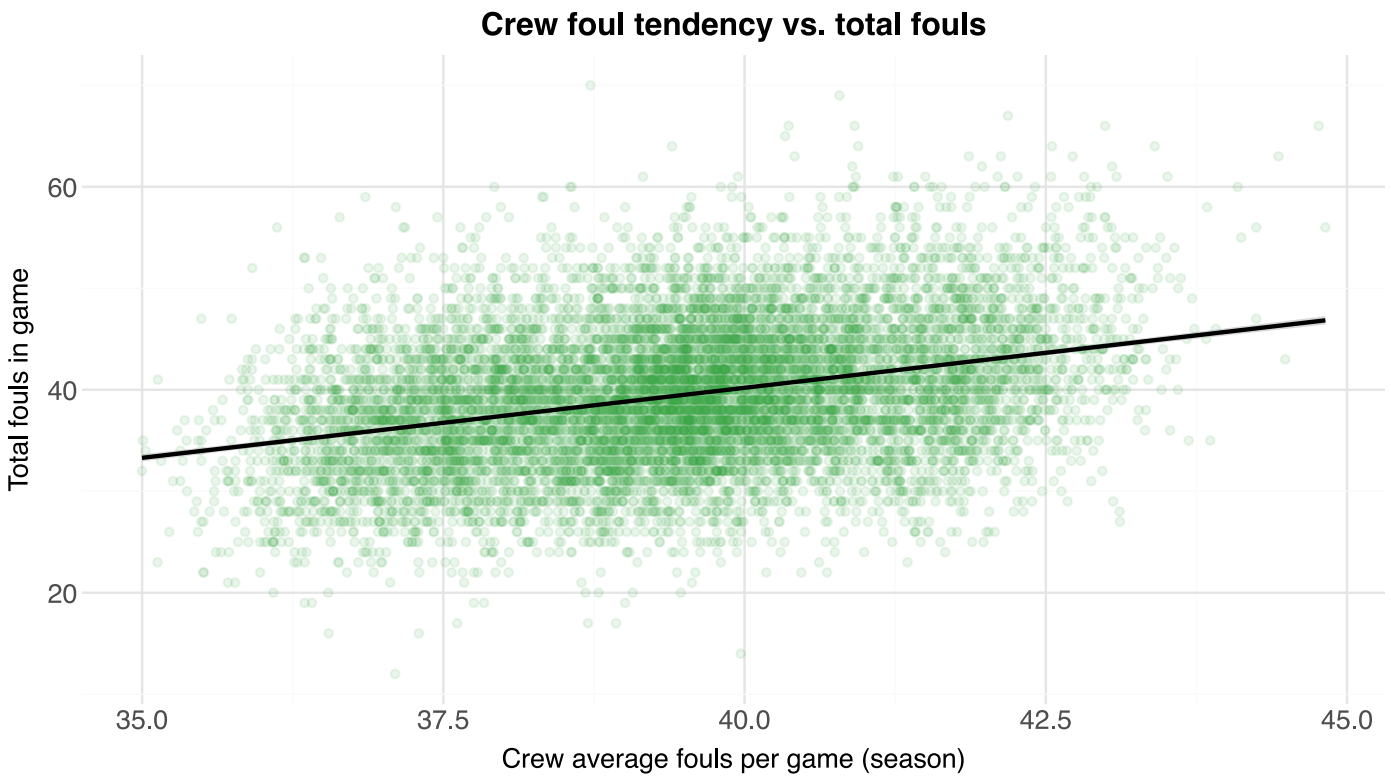
3.4.3 Does the Crew Matter?

```
crewGroups = (gamesClean
  .with_columns(
    crew_group = pl.col('crew_fouls_per_game')
      .qcut(3, labels = ['Low-foul crews', 'Middle crews', 'High-foul crews'])
      .cast(pl.String)
  )
  .group_by('crew_group')
  .agg(avg_fouls = pl.col('total_fouls').mean())
)

(ggplot(crewGroups, aes(x = 'crew_group', y = 'avg_fouls'))
  + geom_col(fill = nbaGreen)
  + geom_text(aes(label = 'avg_fouls'), format_string = '{:.1f}', va = 'bottom')
  + scale_x_discrete(limits = ['Low-foul crews', 'Middle crews', 'High-foul crews'])
  + labs(x = '', y = 'Average total fouls per game',
         title = 'Games with high-foul crews have more fouls')
  + nbaTheme
)
```




```
(ggplot(gamesClean, aes(x = 'crew_fouls_per_game', y = 'total_fouls'))
    + geom_point(alpha = 0.1, color = nbaGreen)
    + geom_smooth(method = 'lm', color = 'black')
    + labs(x = 'Crew average fouls per game (season)', y = 'Total fouls in game',
           title = 'Crew foul tendency vs. total fouls')
    + nbaTheme
)
```



Games officiated by the third of crews with the highest foul tendency average about 5 more fouls than games with the lowest third. The scatterplot shows a clear positive, roughly linear relationship, with a lot of game-to-game spread.

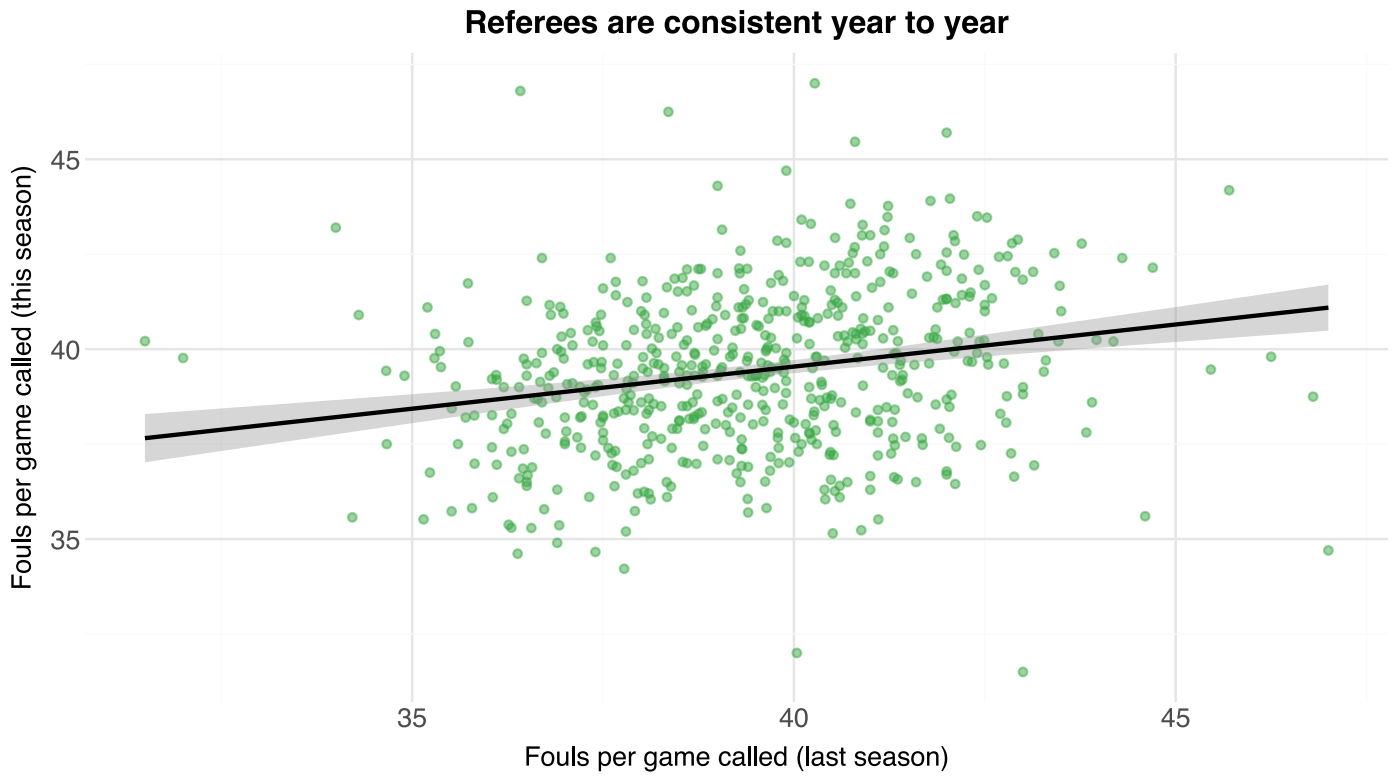
3.4.4 Referees Are Consistent Year to Year

```
refPairs = (refStats
  .select('season', 'referee', 'fouls_per_game')
  .with_columns(start = pl.col('season').str.slice(0, 4).cast(pl.Int64))
)
refPairs = refPairs.join(
  refPairs.with_columns(start = pl.col('start') + 1),
  on = ['referee', 'start'], suffix = '_last'
)

print('Correlation, last season vs. this season:',
      round(refPairs.select(pl.corr('fouls_per_game_last', 'fouls_per_game')).item(), 2))

(ggplot(refPairs, aes(x = 'fouls_per_game_last', y = 'fouls_per_game'))
  + geom_point(color = nbaGreen, alpha = 0.5)
  + geom_smooth(method = 'lm', color = 'black')
  + labs(x = 'Fouls per game called (last season)', y = 'Fouls per game called (this season)',
         title = 'Referees are consistent year to year')
  + nbaTheme
)
```

```
Correlation, last season vs. this season: 0.23
```

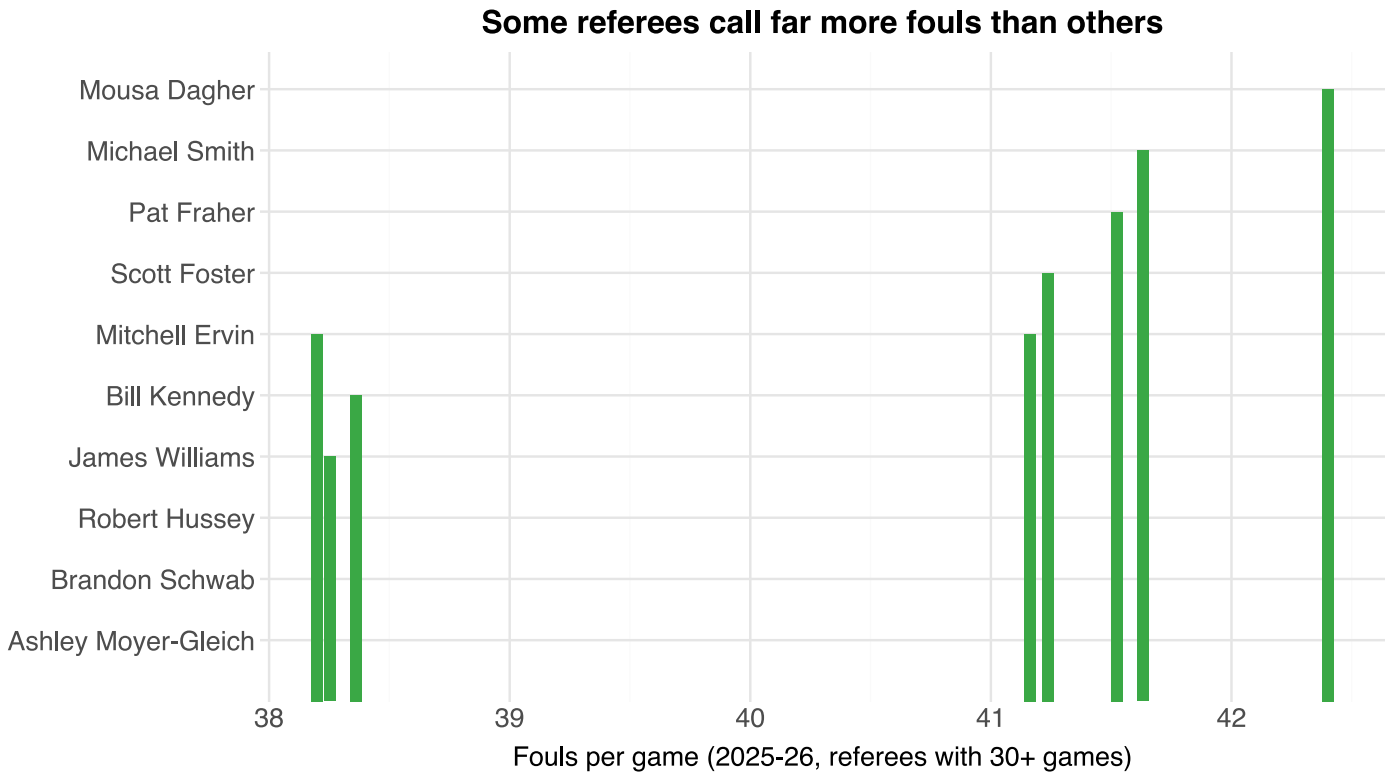


A referee's foul rate last season is positively related to their foul rate this season (correlation about 0.23). The relationship is modest, so past tendencies are a noisy guide, but they do carry forward. This matters for the business use: before a game, only *past* referee data is available, and this plot shows past tendencies are informative.

3.4.5 Some Referees Call Far More Fouls Than Others

```
lastSeason = refStats.filter(pl.col('season') == '2025-2026', pl.col('games') >= 30)
refExtremes = pl.concat([
  lastSeason.sort('fouls_per_game').head(5),
  lastSeason.sort('fouls_per_game').tail(5),
])

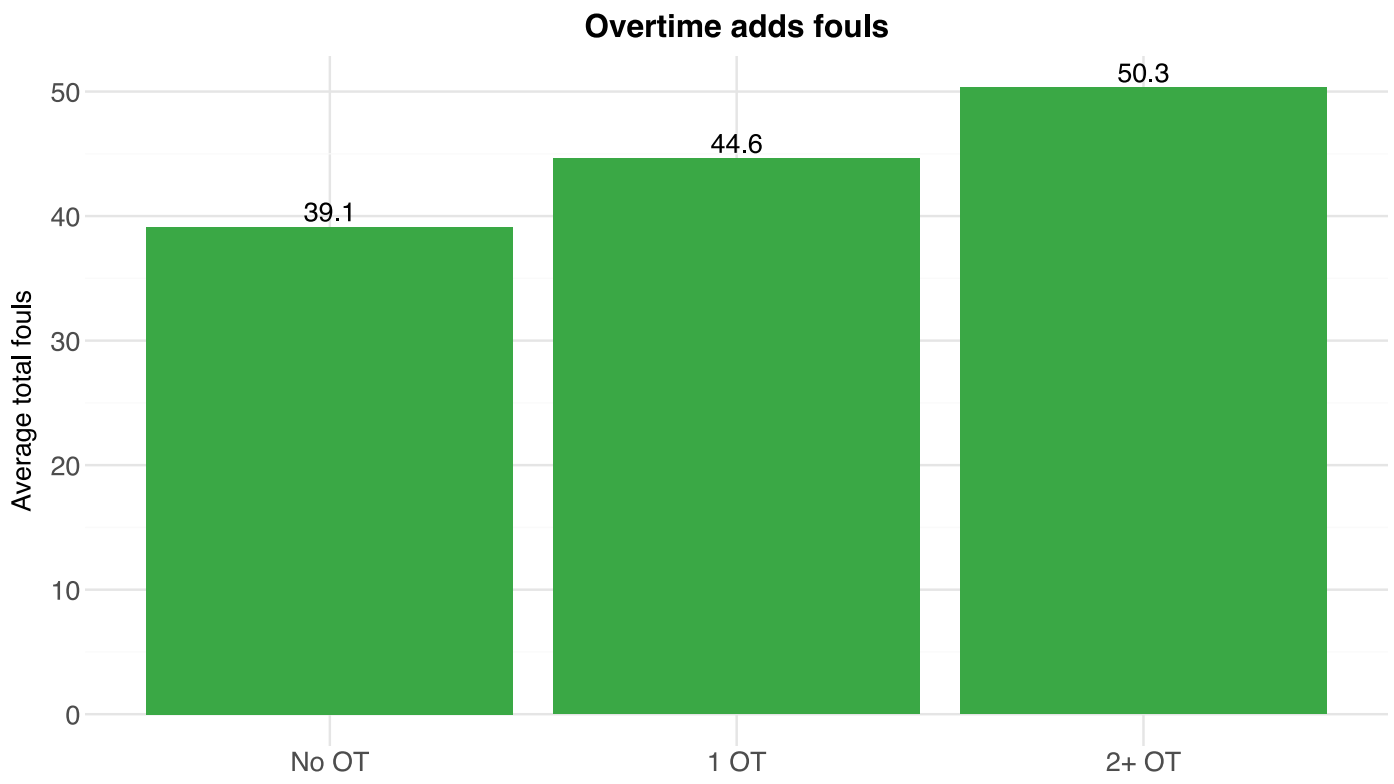
(ggplot(refExtremes, aes(x = 'fouls_per_game', y = 'referee'))
  + geom_col(fill = nbaGreen)
  + scale_y_discrete(limits = refExtremes['referee'].to_list())
  + labs(x = 'Fouls per game (2025-26, referees with 30+ games)', y = '',
         title = 'Some referees call far more fouls than others')
  + nbaTheme
)
```



3.4.6 Overtime Adds Fouls

```
otData = (gamesClean
  .with_columns(ot_group = pl.when(pl.col('overtime') == 0).then(pl.lit('No OT'))
                 .when(pl.col('overtime') == 1).then(pl.lit('1 OT'))
                 .otherwise(pl.lit('2+ OT')))
  .group_by('ot_group')
  .agg(avg_fouls = pl.col('total_fouls').mean(), games = pl.len())
)

(ggplot(otData, aes(x = 'ot_group', y = 'avg_fouls'))
  + geom_col(fill = nbaGreen)
  + geom_text(aes(label = 'avg_fouls'), format_string = '{:.1f}', va = 'bottom')
  + scale_x_discrete(limits = ['No OT', '1 OT', '2+ OT'])
  + labs(x = '', y = 'Average total fouls', title = 'Overtime adds fouls')
  + nbaTheme
)
```



3.4.7 Correlations

```
corrTable = (gamesClean
    .select('total_fouls', 'crew_fouls_per_game', 'crew_experience', 'crew_foul_diff',
            'crew_home_win_pct', 'home_team_fouls', 'away_team_fouls', 'overtime')
    .corr()
    .select(pl.all().round(2))
    .rename({
        'total_fouls': 'total', 'crew_fouls_per_game': 'crew_fouls', 'crew_experience': 'crew_exp',
        'crew_foul_diff': 'crew_diff', 'crew_home_win_pct': 'crew_hwp', 'home_team_fouls': 'home_fouls',
        'away_team_fouls': 'away_fouls', 'overtime': 'OT',
    })
)
with pl.Config(tbl_cols = 10):
    display(corrTable.insert_column(0, pl.Series('variable', corrTable.columns)))
```

variable	total	crew_fouls	crew_exp	crew_diff	crew_hwp	home_fouls	away_fouls	OT
str	f64	f64	f64	f64	f64	f64	f64	f64
"total"	1.0	0.34	−0.04	0.05	0.05	0.3	0.29	0.21
"crew_fouls"	0.34	1.0	−0.12	0.19	0.24	0.48	0.48	0.04
"crew_exp"	−0.04	−0.12	1.0	0.04	0.01	−0.06	−0.07	−0.02
"crew_diff"	0.05	0.19	0.04	1.0	0.24	0.07	0.15	−0.01
"crew_hwp"	0.05	0.24	0.01	0.24	1.0	0.12	0.15	0.0
"home_fouls"	0.3	0.48	−0.06	0.07	0.12	1.0	0.25	0.02
"away_fouls"	0.29	0.48	−0.07	0.15	0.15	0.25	1.0	0.0
"OT"	0.21	0.04	−0.02	−0.01	0.0	0.02	0.0	1.0

`crew_fouls_per_game` has the strongest correlation with total fouls (0.34), followed by the two team foul averages (about 0.3) and overtime (0.21). Crew experience, foul differential, and home-team win % have almost no correlation with total fouls. The crew and team foul averages are moderately correlated with each other (0.48), so we check multicollinearity in the Reconcile section.

3.5 Data Limitations

- **Leakage in season averages.** Referee and team foul averages are computed over the full season, including the game being predicted (see *Comparing Real Data to Ideal Data*).
- **Repeated units.** The same referees and teams appear in many games. Each row is a different game, but games that share a referee or team are not fully independent.
- **Crew averaging** ignores roles (crew chief vs. member) and how referees interact.
- **Proxy variables.** Team season foul averages stand in for team style, roster, and strategy.
- **Overtime is unknown before tip-off**, so it is a control, not a pre-game input.
- **Scope.** Regular-season games from 2017–18 to 2025–26 only. Playoff and preseason officiating may differ.

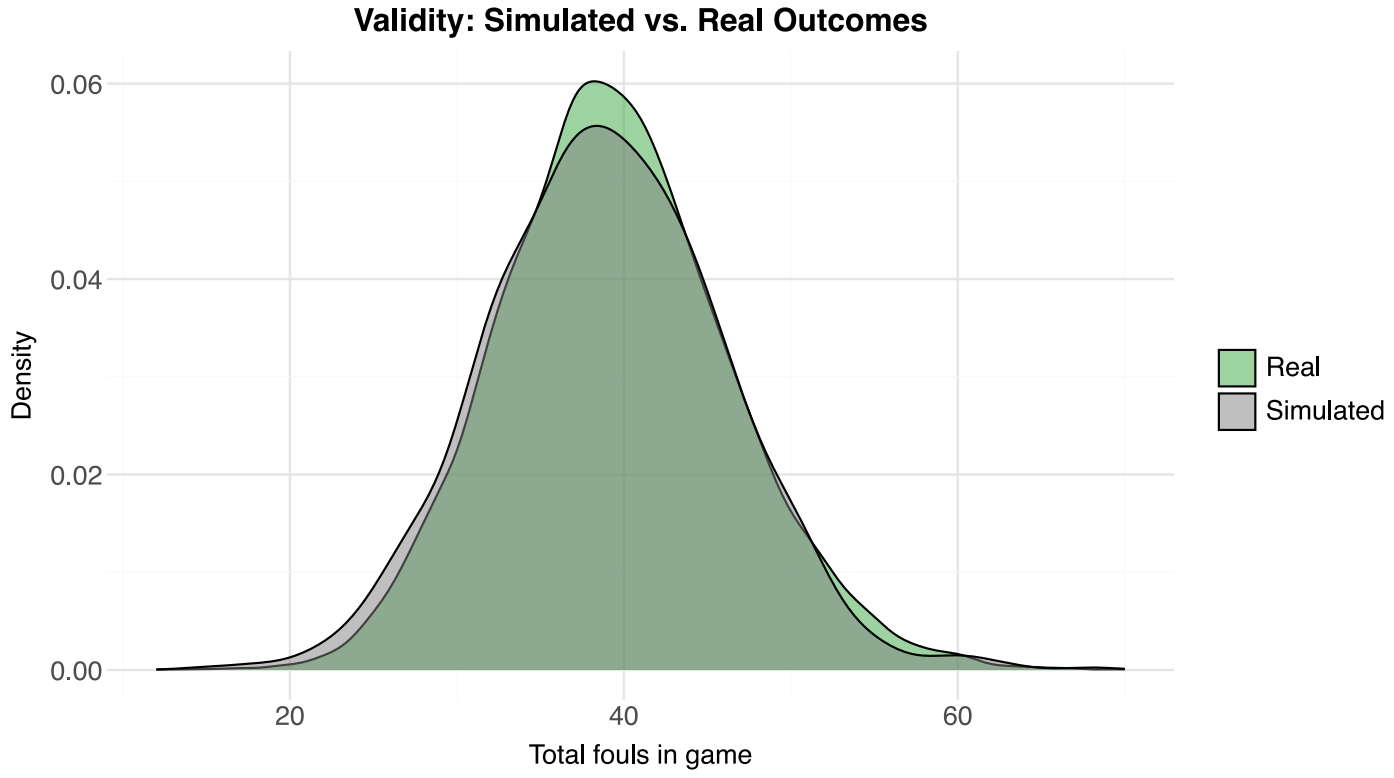
4 Reconcile

4.1 Validity: Simulated vs. Real Outcomes

Comparing our simulated outcome to the real outcome checks whether our data story produces data that looks like real games.

```
validityDf = pl.concat([
    simData.select('total_fouls').with_columns(source = pl.lit('Simulated')),
    gamesClean.select(pl.col('total_fouls').cast(pl.Float64)).with_columns(source = pl.lit('Real')),
])

(ggplot(validityDf, aes(x = 'total_fouls', fill = 'source'))
  + geom_density(alpha = 0.5)
  + scale_fill_manual(values = {'Real': nbaGreen, 'Simulated': 'grey'})
  + labs(x = 'Total fouls in game', y = 'Density', fill = '',
         title = 'Validity: Simulated vs. Real Outcomes')
  + nbaTheme
)
```



The simulated and real distributions have a similar center and spread, so the data story is a reasonable description of how real foul totals are generated. The real data is slightly more peaked with a longer right tail, consistent with real-world factors (season rule changes, blowouts) that the simple simulation leaves out.

4.2 Feature Engineering

- **No log transformation.** Total fouls are approximately bell-shaped with only a slight right tail, and the continuous predictors are roughly symmetric. A log transform would make the coefficients harder to interpret without a clear gain, so we keep the original scale (fouls).
- **Dummy coding.** `season` is dummy coded with **2017–18 as the reference level**, and `late_season` is converted from True/False to 1/0.
- **Train/test split.** We randomly split games 80% / 20% (`random_state = 42`). Models are fit on the training data and compared on the held-out test data.

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import root_mean_squared_error, mean_squared_error

X = gamesClean.select(
    'crew_fouls_per_game', 'crew_experience', 'crew_foul_diff', 'crew_home_win_pct',
    'home_team_fouls', 'away_team_fouls', 'overtime', 'late_season', 'season'
)
y = gamesClean.select('total_fouls')

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state = 42)

## Same steps for train and test data
def prepX(df):
    out = (df
        .with_columns(pl.col('late_season').cast(pl.Int64))
        .to_dummies(columns = ['season'], drop_first = False)
        .select(pl.exclude('season_2017-2018'))       ## 2017-18 is the reference season
        .rename(lambda c: c.replace('-', '_'))
    )
    return sm.add_constant(out.to_pandas())

X_train = prepX(X_train)
X_test = prepX(X_test).reindex(columns = X_train.columns, fill_value = 0)
y_train = y_train.to_pandas()['total_fouls']
y_test = y_test.to_pandas()['total_fouls']

print('Training games:', len(X_train), '| Test games:', len(X_test))
```

```
Training games: 8583 | Test games: 2146
```

4.3 Fitting the Selected Model

The assumption checks below use the final model (all predictors, including season), fit on the training data. Model comparisons are in the Evaluate section.

```
fit_05 = sm.OLS(y_train, X_train).fit()

## Training data with residuals and fitted values, in date order
trainDiag = pl.DataFrame({
    'game_date': gamesClean['game_date'].to_numpy()[X_train.index],
    'fitted': fit_05.fittedvalues.to_numpy(),
    'residuals': fit_05.resid.to_numpy(),
    'dffits': fit_05.get_influence().dffits[0],
}).sort('game_date').with_row_index('observation')
```

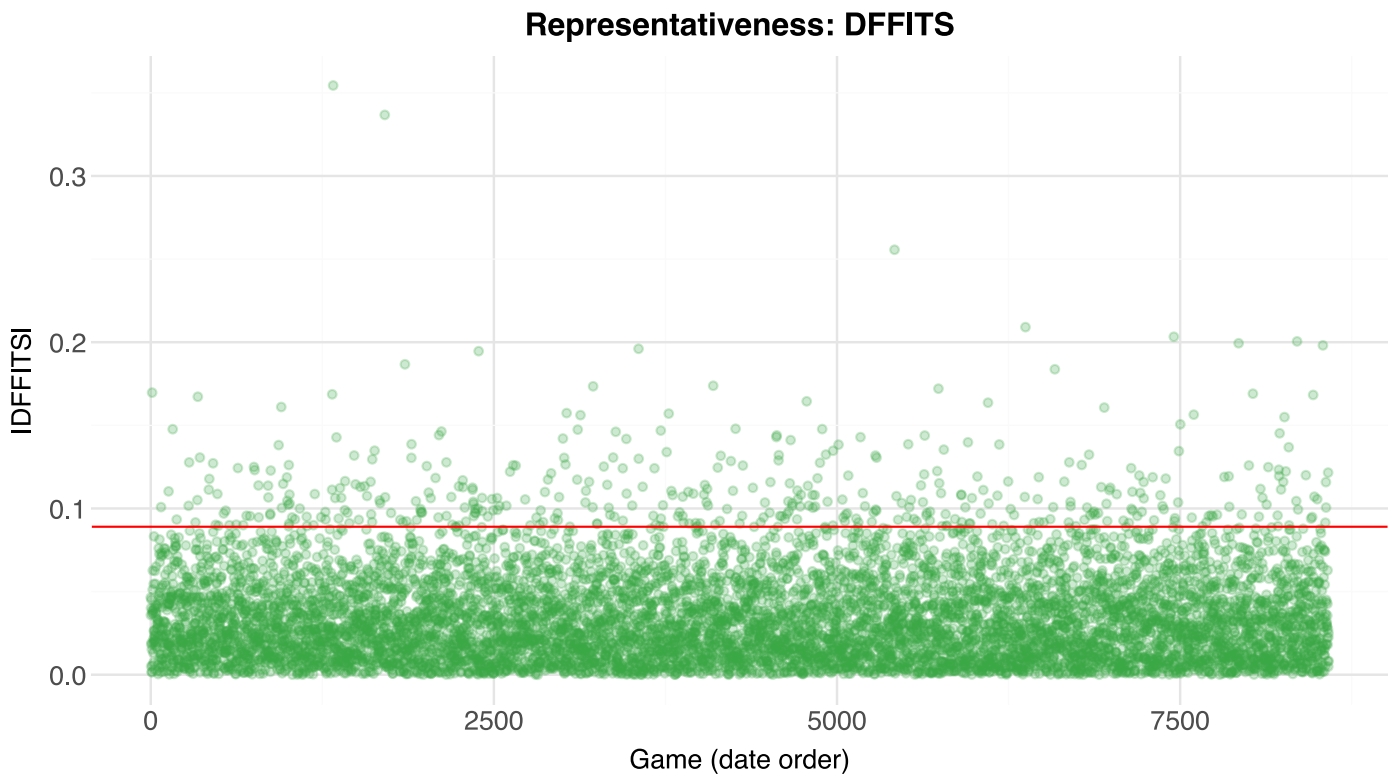
4.4 Representativeness: Influential Games

```
n, p = len(X_train), len(fit_05.params)
dffitsCutoff = 1 if n <= 30 else 2 * np.sqrt(p / n)
trainDiag = trainDiag.with_columns(abs_dffits = pl.col('dffits').abs())

print('DFFITS cutoff:', round(dffitsCutoff, 3))
print('Games above cutoff:', (trainDiag['abs_dffits'] > dffitsCutoff).sum(),
      f"({(trainDiag['abs_dffits'] > dffitsCutoff).mean():.1%} of training games)")

(ggplot(trainDiag, aes(x = 'observation', y = 'abs_dffits'))
  + geom_point(alpha = 0.25, color = nbaGreen)
  + geom_hline(yintercept = dffitsCutoff, color = 'red')
  + labs(x = 'Game (date order)', y = '|DFFITS|', title = 'Representativeness: DFFITS')
  + nbaTheme
)
```

```
DFFITS cutoff: 0.089
Games above cutoff: 425 (5.0% of training games)
```

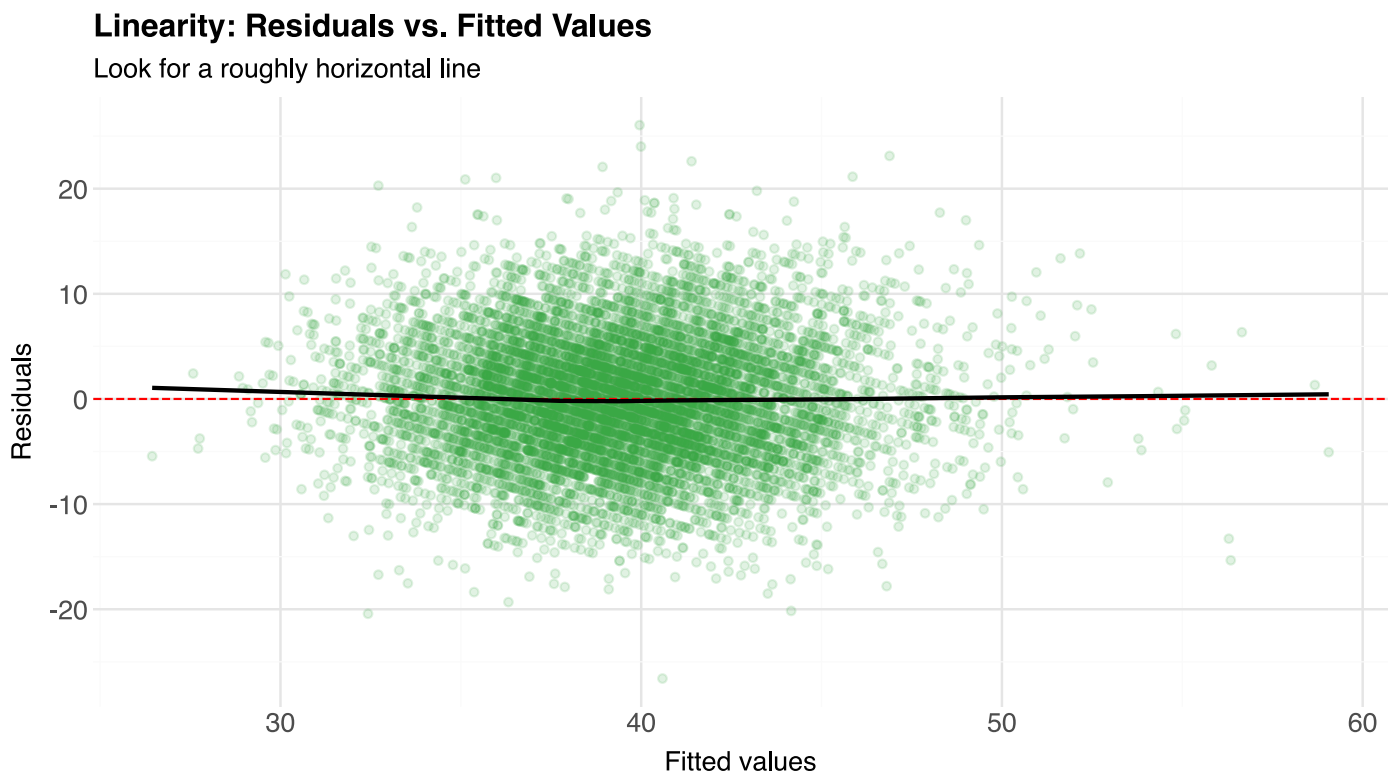


About 5% of training games are above the DFFITS cutoff, which is expected with more than 8,000 games and a small cutoff (about 0.09). We inspected the extreme games earlier (very low and very high foul totals, multi-overtime games) and found them to be real games rather than errors. Because they are legitimate outcomes of the process we want to predict, we keep them.

4.5 Linearity

```
(ggplot(trainDiag, aes(x = 'fitted', y = 'residuals'))
  + geom_point(alpha = 0.15, color = nbaGreen)
  + geom_hline(yintercept = 0, color = 'red', linetype = '--')
  + geom_smooth(method = 'lowess', color = 'black')
  + labs(x = 'Fitted values', y = 'Residuals',
         title = 'Linearity: Residuals vs. Fitted Values',
```

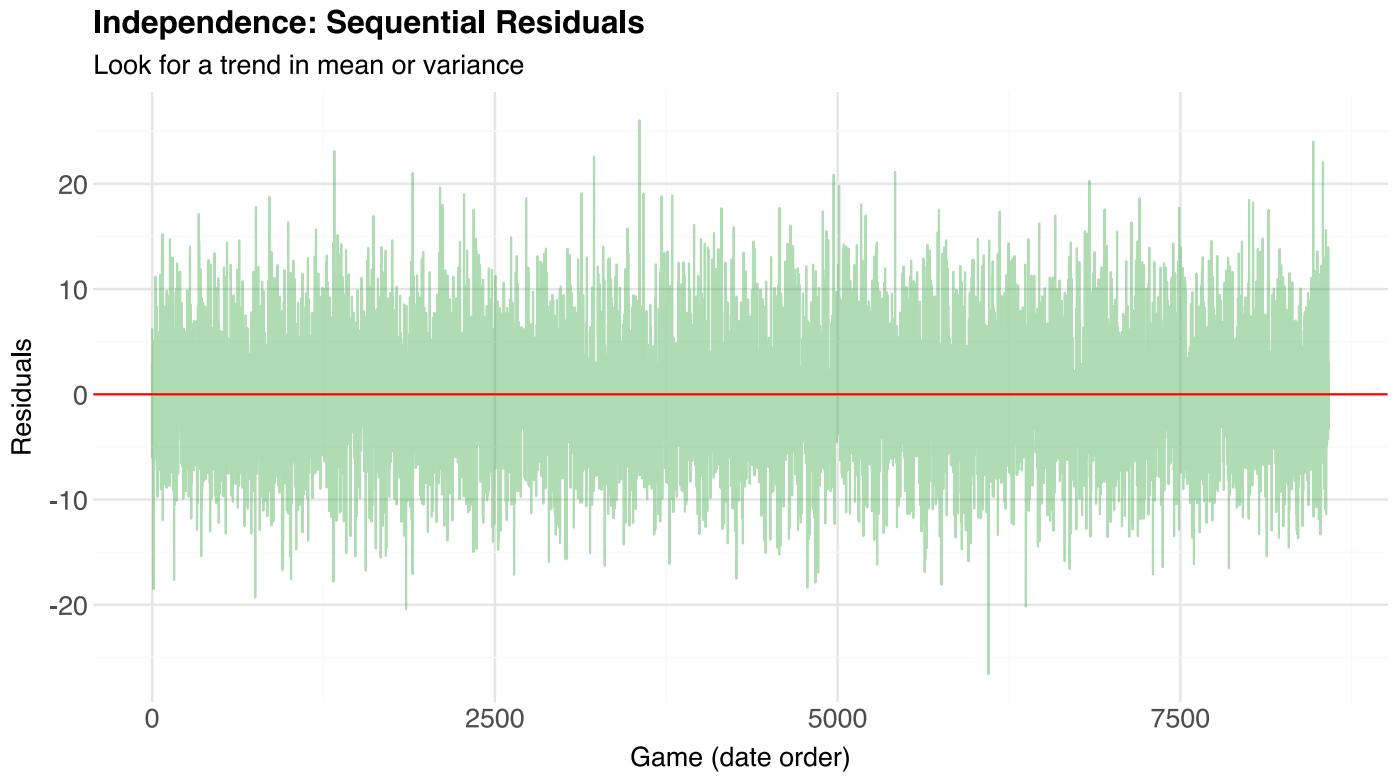
```
        subtitle = 'Look for a roughly horizontal line')
  + nbaTheme
)
```



The lowess line is almost flat at zero across the range of fitted values. There is no curved pattern left in the residuals, so the linearity assumption is satisfied.

4.6 Independence

```
(ggplot(trainDiag, aes(x = 'observation', y = 'residuals'))
  + geom_line(alpha = 0.4, color = nbaGreen)
  + geom_hline(yintercept = 0, color = 'red')
  + labs(x = 'Game (date order)', y = 'Residuals',
         title = 'Independence: Sequential Residuals',
         subtitle = 'Look for a trend in mean or variance')
  + nbaTheme
)
```



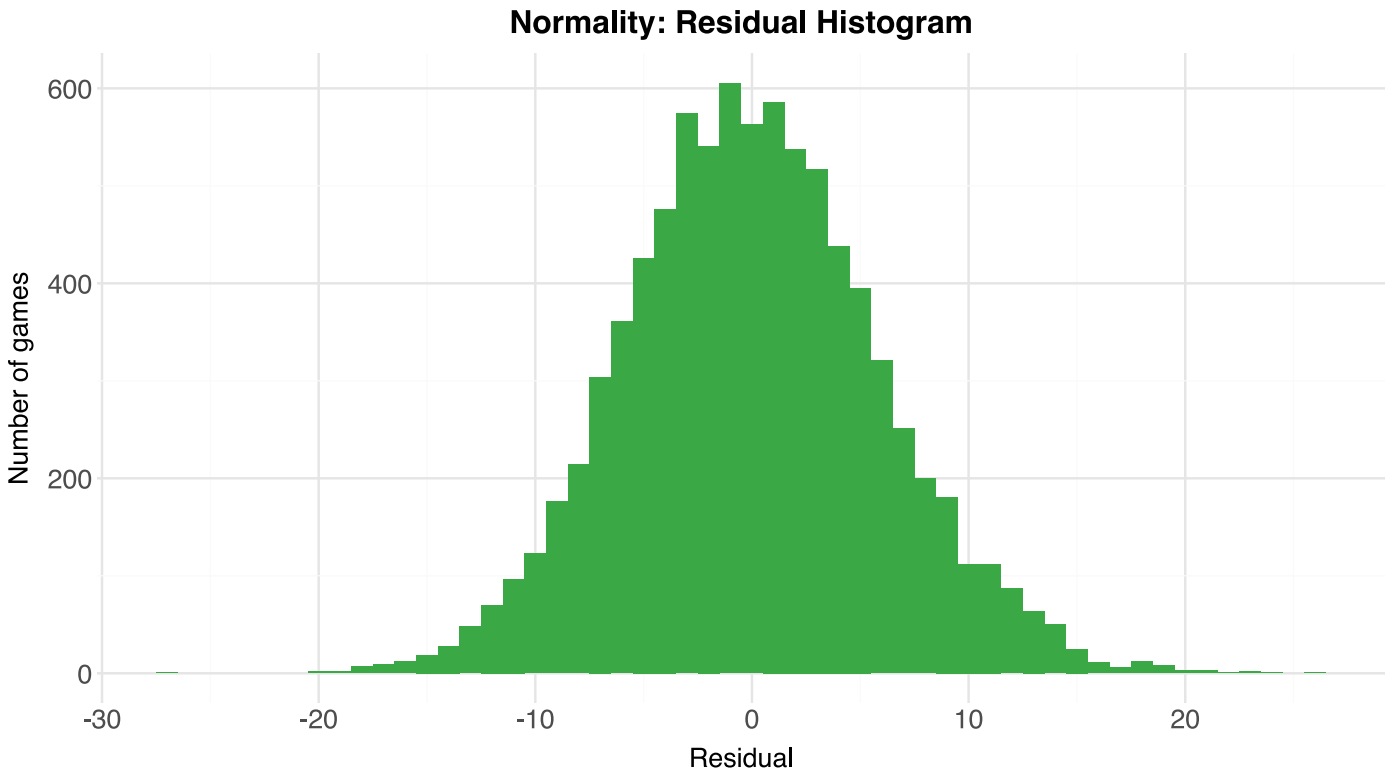
Ordered by game date, the residuals show no trend in the mean or the spread. Including season dummies removes the season-to-season shifts in foul levels. Independence is reasonably satisfied, with the caveat from *Data Limitations* that games sharing referees or teams are related.

4.7 Constant Variance

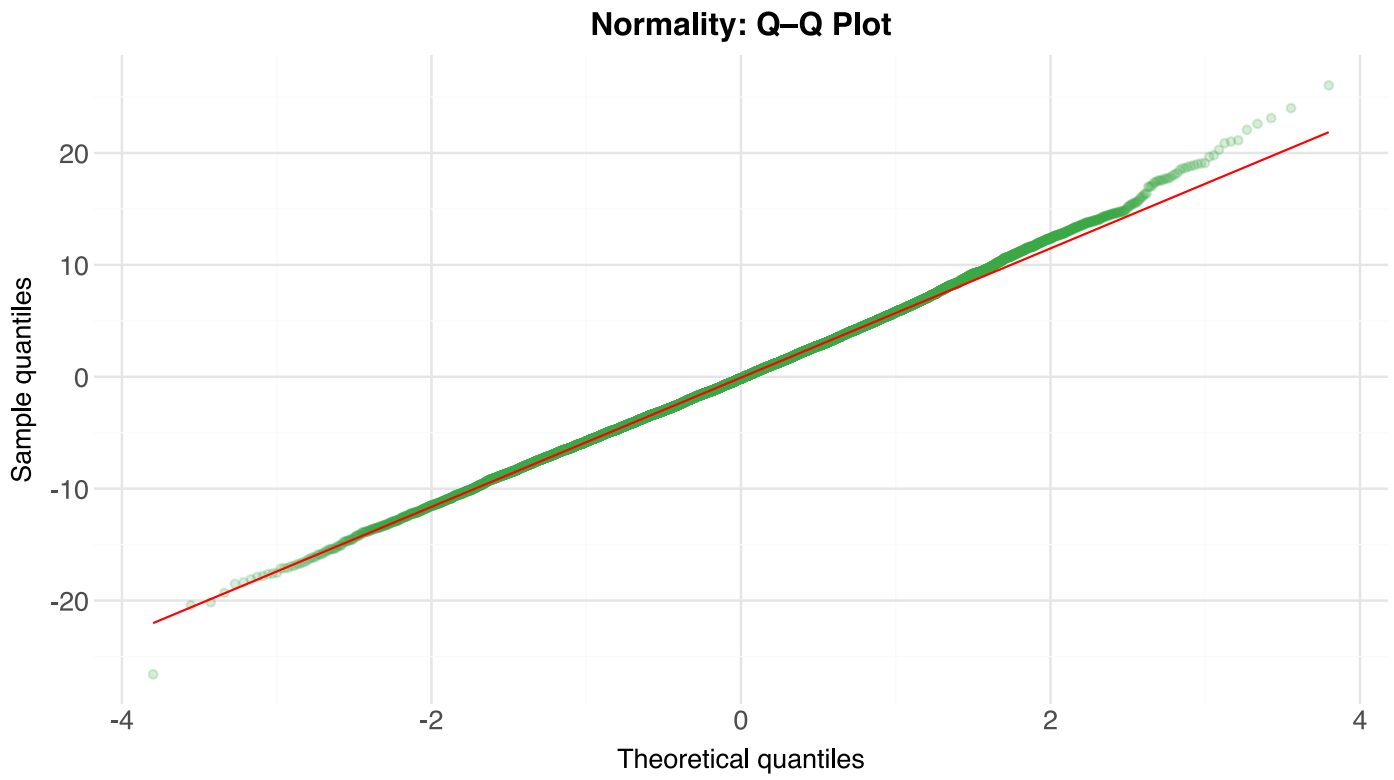
Using the residuals vs. fitted plot above, the vertical spread of residuals is similar across most fitted values, with no funnel shape. There are fewer games at the very highest predictions (mostly overtime games), which makes the spread look slightly uneven there. Constant variance is approximately satisfied.

4.8 Normality

```
(ggplot(trainDiag, aes(x = 'residuals'))
  + geom_histogram(binwidth = 1, fill = nbaGreen)
  + labs(x = 'Residual', y = 'Number of games', title = 'Normality: Residual Histogram')
  + nbaTheme
)
```




```
(ggplot(trainDiag, aes(sample = 'residuals'))
  + stat_qq(alpha = 0.2, color = nbaGreen)
  + stat_qq_line(color = 'red')
  + labs(x = 'Theoretical quantiles', y = 'Sample quantiles', title = 'Normality: Q–Q Plot')
  + nbaTheme
)
```



The residual histogram is bell-shaped and centered at zero. Most Q–Q points follow the reference line; the upper tail bends slightly above the line, meaning a few games had more fouls than a normal distribution would predict. With more than 8,000 training games, this small tail departure has little effect on the coefficient estimates and intervals. Normality is approximately satisfied.

4.9 Identifiability

```
from statsmodels.stats.outliers_influence import variance_inflation_factor

## Rank check
print('Matrix rank:', np.linalg.matrix_rank(X_train.to_numpy()), '| Number of columns:', X_train.shape[1])

## VIF (computed with the constant included, constant row not shown)
vifTable = pl.DataFrame({
    'predictor': X_train.columns[1:].tolist(),
    'VIF': [variance_inflation_factor(X_train.to_numpy().astype(float), i) for i in range(1, X_train.shape[1])],
}).sort('VIF', descending = True)
print('Mean VIF:', round(vifTable['VIF'].mean(), 2))
with pl.Config(tbl_rows = 20):
    display(vifTable.with_columns(pl.col('VIF').round(2)))
```

```
Matrix rank: 17 | Number of columns: 17
Mean VIF: 2.01
```

predictor	VIF
str	f64
"crew_fouls_per_game"	4.91
"season_2024_2025"	3.33
"season_2023_2024"	3.04

predictor	VIF
str	f64
"season_2018_2019"	2.63
"season_2019_2020"	2.28
"season_2020_2021"	2.07
"season_2021_2022"	2.03
"season_2025_2026"	1.87
"season_2022_2023"	1.8
"away_team_fouls"	1.38
"home_team_fouls"	1.38
"crew_home_win_pct"	1.27
"crew_foul_diff"	1.18
"crew_experience"	1.04
"overtime"	1.0
"late_season"	1.0

The design matrix has full rank, so every coefficient can be estimated. The largest VIF is about 4.9 (`crew_fouls_per_game`), well below 10, and the mean VIF is about 2, not much more than 1. Multicollinearity is not a problem. The moderate correlation between crew and team foul averages (0.48) does not inflate the coefficient variances enough to affect interpretation. Identifiability is satisfied.

Summary: all assumptions are satisfied or approximately satisfied, and no remedies (transformations or dropping predictors) are needed.

5 Evaluate

5.1 Models Compared

We compared three models, all fit on the same training data and tested on the same held-out games:

- **Model A — Teams only:** team foul averages, game context, and season. *No referee variables.* This answers "how well can we do without scouting the referees?"
- **Model B — No season:** referee and team variables plus game context, but no season dummies.
- **Model C — Full model:** all variables (selected).

```
crewCols = ['crew_fouls_per_game', 'crew_experience', 'crew_foul_diff', 'crew_home_win_pct']
seasonCols = [c for c in X_train.columns if c.startswith('season_')]

modelSpecs = {
    'A: Teams only (no crew)': [c for c in X_train.columns if c not in crewCols],
    'B: No season': [c for c in X_train.columns if c not in seasonCols],
    'C: Full model': list(X_train.columns),
}

compareRows = []
for name, cols in modelSpecs.items():
    fit = sm.OLS(y_train, X_train[cols]).fit()
    pred = fit.predict(X_test[cols])
    compareRows.append({
        'model': name,
        'predictors': len(cols) - 1,
        'R2': round(fit.rsquared, 3),
        'adj_R2': round(fit.rsquared_adj, 3),
        'test_RMSE': round(root_mean_squared_error(y_test, pred), 3),
```

```
        'test_MSE': round(mean_squared_error(y_test, pred), 2),
    })
compareTable = pl.DataFrame(compareRows)
compareTable
```

model	predictors	R2	adj_R2	test_RMSE	test_MSE
str	i64	f64	f64	f64	f64
"A: Teams only (no crew)"	12	0.184	0.183	6.182	38.21
"B: No season"	8	0.199	0.198	6.099	37.2
"C: Full model"	16	0.265	0.264	5.846	34.17

The full model (C) has the highest adjusted R^2 and the lowest test RMSE and MSE, so we select it. Removing the referee variables (Model A) makes predictions worse on new games, which supports our main finding that the crew adds information beyond the teams. Removing season (Model B) also hurts fit, because league-wide foul levels change from year to year. The difference in test RMSE between models is modest in absolute terms (fractions of a foul), which reflects the large amount of game-to-game randomness in fouls that no pre-game model can explain.

5.2 Frequentist Results (OLS)

```
ciTable = fit_05.conf_int()
ols_results = pd.DataFrame({
    'Estimate': fit_05.params,
    'CI Lower': ciTable[0],
    'CI Upper': ciTable[1],
    'p-value': fit_05.pvalues,
}).round(3)
print(ols_results)
print(f'\nR-squared: {fit_05.rsquared:.3f} | Adj. R-squared: {fit_05.rsquared_adj:.3f}')
print(f'F-statistic: {fit_05.fvalue:.1f} | Prob (F-statistic): {fit_05.f_pvalue:.3g}')
```

```
                    Estimate  CI Lower  CI Upper  p-value
const                -111.559  -118.940  -104.177    0.000
crew_fouls_per_game     2.592     2.427     2.757    0.000
crew_experience         0.004    -0.025     0.034    0.770
crew_foul_diff          0.091    -0.204     0.387    0.546
crew_home_win_pct       0.224    -2.826     3.275    0.885
home_team_fouls         1.211     1.107     1.314    0.000
away_team_fouls         1.185     1.082     1.288    0.000
overtime                4.849     4.375     5.324    0.000
late_season            -1.562    -1.985    -1.140    0.000
season_2018_2019       -5.521    -6.157    -4.885    0.000
season_2019_2020       -4.745    -5.382    -4.108    0.000
season_2020_2021        3.642     3.041     4.243    0.000
season_2021_2022        1.707     1.145     2.268    0.000
season_2022_2023       -0.243    -0.768     0.282    0.364
season_2023_2024        6.733     6.048     7.418    0.000
season_2024_2025        7.571     6.852     8.291    0.000
season_2025_2026        0.487    -0.045     1.019    0.073

R-squared: 0.265 | Adj. R-squared: 0.264
F-statistic: 193.1 | Prob (F-statistic): 0
```

5.3 Bayesian Results

We also fit the same model as a Bayesian linear regression with flat (uniform) priors on all coefficients and an exponential prior on σ. With flat priors and a large sample, we expect the Bayesian results to closely match OLS.

```
import pytensor
pytensor.config.cxx = ""
import bambi as bmb
import arviz as az

predictors = [c for c in X_train.columns if c != 'const']

## Combine outcome and predictors for bambi
nba_train = pd.concat([y_train, X_train.drop(columns = 'const')], axis = 1)

ba_model = bmb.Model('total_fouls ~ ' + ' + '.join(predictors), data = nba_train)

## Flat priors on coefficients, exponential prior on sigma
priors_dict = {
  'Intercept': bmb.Prior('Uniform', lower = -100, upper = 100),
  **{term: bmb.Prior('Uniform', lower = -100, upper = 100) for term in predictors},
  'sigma': bmb.Prior('Exponential', lam = 1)
}
ba_model.set_priors(priors = priors_dict)

ba_fit = ba_model.fit(chains = 4, cores = 1, random_seed = 123)
```

```
baSum = az.summary(ba_fit, ci_kind = 'hdi', ci_prob = 0.95)
baSum[['mean', 'sd', baSum.columns[2], baSum.columns[3], 'r_hat']].round(3)
```

	mean	sd	hdi95_lb	hdi95_ub	r_hat
sigma	5.909	0.045	5.8	6	1.00
Intercept	−111.5	3.8	−120	−100	1.00
crew_fouls_per_game	2.591	0.086	2.4	2.8	1.00
crew_experience	0.0043	0.0155	−0.026	0.035	1.00
crew_foul_diff	0.089	0.151	−0.2	0.38	1.00
crew_home_win_pct	0.2	1.57	−3	3.3	1.00
home_team_fouls	1.21	0.052	1.1	1.3	1.00
away_team_fouls	1.186	0.052	1.1	1.3	1.00
overtime	4.848	0.247	4.4	5.3	1.00
late_season	−1.562	0.215	−2	−1.1	1.00
season_2018_2019	−5.52	0.321	−6.2	−4.9	1.00
season_2019_2020	−4.75	0.318	−5.3	−4.1	1.00
season_2020_2021	3.63	0.307	3	4.3	1.00
season_2021_2022	1.7	0.287	1.1	2.3	1.00
season_2022_2023	−0.25	0.269	−0.77	0.27	1.00
season_2023_2024	6.72	0.35	6	7.4	1.00
season_2024_2025	7.56	0.38	6.8	8.3	1.00
season_2025_2026	0.48	0.27	−0.041	1	1.00

All $\hat{R}$ values are 1.00, which means the four chains agree and the sampler converged. Because MCMC uses random draws, Bayesian values may differ slightly in the last digit between renders. (Bambi applies the intercept prior to the intercept of centered predictors, so the reported intercept can fall outside the ±100 prior range.)

5.4 Overall Model Fit

- R^2 **= 0.265 (adjusted 0.264).** The model explains about 26.5% of the variation in total fouls in the training games. The remaining variation is game-to-game randomness (game flow, blowouts, specific matchups, player-level foul trouble) that is not known before tip-off.
- **F-statistic.** The p-value is essentially zero, so there is strong evidence that at least one predictor helps explain total fouls.
- **Test RMSE ≈ 5.9 fouls.** On games the model did not see, predictions are off by about 6 fouls on average.

An R^2 of 0.265 is modest, but our objective is not to predict every game exactly. It is to tell coaches whether to expect a higher- or lower-foul game than usual, and to measure how much the crew matters.

5.5 Interpreting the Parameter Estimates

All slope interpretations hold all other variables fixed. Because the OLS confidence intervals and Bayesian credible intervals were nearly identical, we report both below and interpret them separately: a **95% confidence interval** comes from a procedure that captures the true parameter in 95% of repeated samples, while a **95% credible interval** means there is a 0.95 probability the parameter is in the interval, given the data, model, and priors.

Intercept (−111.56). This is the predicted total fouls when every predictor is 0 (for example, a crew that calls 0 fouls per game), which is impossible. We do not interpret it.

Crew variables

- `crew_fouls_per_game` **(2.59).** For each additional foul per game in the crew's usual (season average) foul rate, the game has about 2.6 more total fouls. 95% CI: 2.43 to 2.76; 95% credible interval: 2.4 to 2.8. Both exclude zero ($p < 0.001$), so the effect is statistically significant. It is also practically significant: the effect is more than twice the size of either team's effect (about 1.2). A one-unit change in a three-referee crew average is a large change in crew terms; within the 2025–26 season, moving from a low-foul crew (10th percentile) to a high-foul crew (90th percentile) adds about 4 predicted fouls.
- `crew_experience` **(0.004).** One more year of average crew experience changes total fouls by almost nothing. 95% CI: −0.025 to 0.034 ($p = 0.770$); credible interval: −0.024 to 0.034. Not statistically significant.
- `crew_foul_diff` **(0.091).** Crews that call more fouls on road teams than home teams do not clearly call more or fewer fouls in total. 95% CI: −0.204 to 0.387 ($p = 0.546$); credible interval: −0.22 to 0.38. Not statistically significant.
- `crew_home_win_pct` **(0.224).** This is a proportion, so one unit is a change from 0% to 100% home wins. Even this extreme change has no clear effect. 95% CI: −2.83 to 3.28 ($p = 0.885$); credible interval: −2.8 to 3.3. Not statistically significant.

These three variables are kept as controls. Learning that experience and home bias do not change the total number of fouls is itself useful: what matters about a crew is its foul rate, not its experience or home lean.

Team variables

- `home_team_fouls` **(1.21).** If the home team averages 1 more foul per game, the game has about 1.2 more total fouls. 95% CI: 1.11 to 1.31; credible interval: 1.1 to 1.3. Statistically significant.
- `away_team_fouls` **(1.19).** If the away team averages 1 more foul per game, the game has about 1.2 more total fouls. 95% CI: 1.08 to 1.29; credible interval: 1.1 to 1.3. Statistically significant.

Game context

- `overtime` **(4.85).** Each overtime period adds about 4.8 total fouls, consistent with five extra minutes of play. 95% CI: 4.38 to 5.32; credible interval: 4.4 to 5.3. Statistically significant.
- `late_season` **(−1.56).** Games in the last two weeks of the season have about 1.6 fewer total fouls than other games. 95% CI: −1.99 to −1.14; credible interval: about −2.0 to −1.1. Statistically significant. This may reflect resting players and less competitive games.

Season dummies (reference level: 2017–18)

Season	Estimate	95% CI	Significant?
2018–19	−5.52	−6.16 to −4.89	Yes ($p < 0.001$)
2019–20	−4.75	−5.38 to −4.11	Yes ($p < 0.001$)
2020–21	3.64	3.04 to 4.24	Yes ($p < 0.001$)
2021–22	1.71	1.15 to 2.27	Yes ($p < 0.001$)
2022–23	−0.24	−0.77 to 0.28	No ($p = 0.364$)

Season	Estimate	95% CI	Significant?
2023–24	6.73	6.05 to 7.42	Yes (p < 0.001)
2024–25	7.57	6.85 to 8.29	Yes (p < 0.001)
2025–26	0.49	−0.05 to 1.02	No (p = 0.073)

The season estimates are *adjusted* differences from 2017–18 after accounting for the crew and team averages. Because crew and team averages already move with league-wide foul levels each season, these dummies do not match the raw season boxplots (for example, 2023–24 has fewer raw fouls but a positive adjusted estimate). They should be read as season adjustments, not as raw season averages. Bayesian credible intervals agreed with OLS in direction and significance for every season.

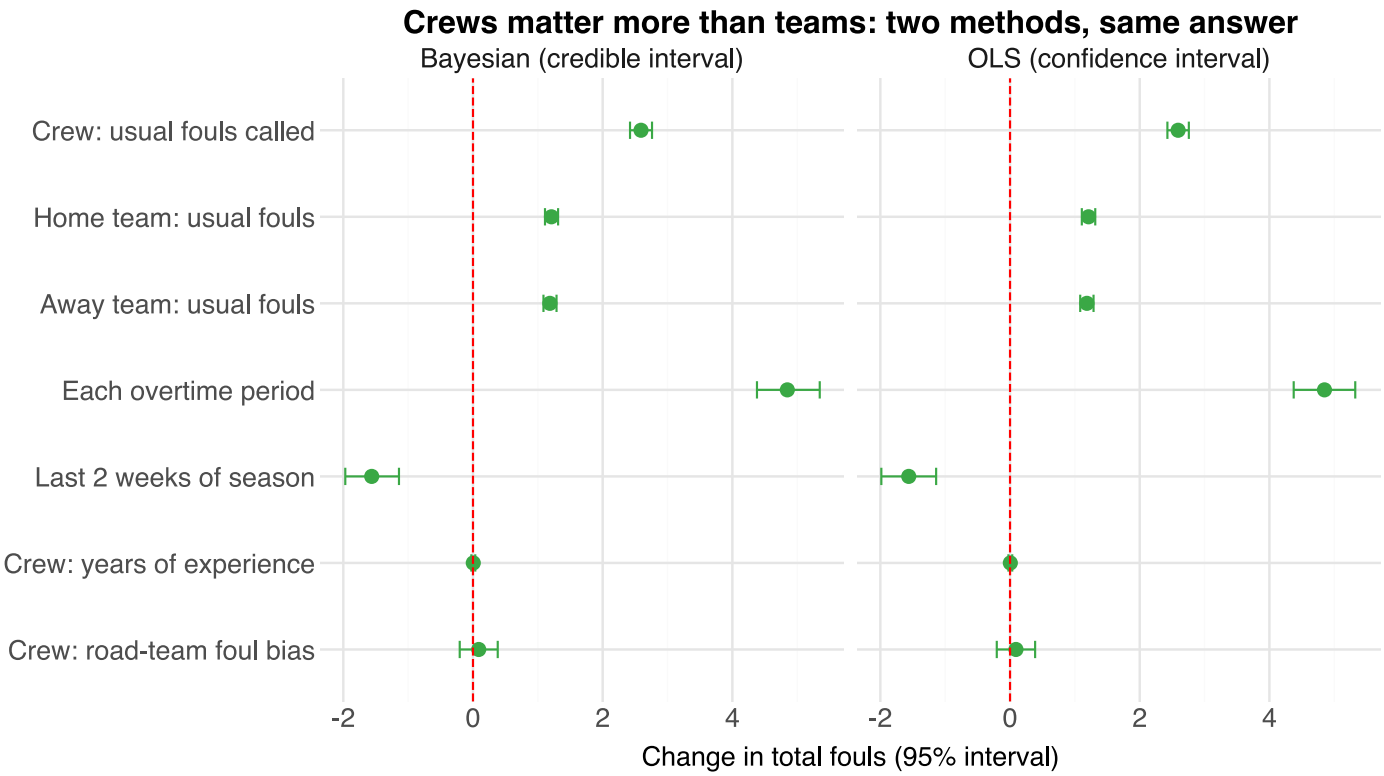
5.6 Visualizing the Interval Estimates

```
niceNames = {
  'crew_fouls_per_game': 'Crew: usual fouls called',
  'home_team_fouls': 'Home team: usual fouls',
  'away_team_fouls': 'Away team: usual fouls',
  'overtime': 'Each overtime period',
  'late_season': 'Last 2 weeks of season',
  'crew_experience': 'Crew: years of experience',
  'crew_foul_diff': 'Crew: road-team foul bias',
  'crew_home_win_pct': 'Crew: home-team win %',
}
plotOrder = list(niceNames.values())[::-1]

olsDf = pl.DataFrame({
  'term': fit_05.params.index.tolist(),
  'coef': fit_05.params.tolist(),
  'low': ciTable[0].tolist(),
  'high': ciTable[1].tolist(),
})
bayesDf = pl.DataFrame({
  'term': baSum.index.tolist(),
  'coef': baSum.iloc[:, 0].tolist(),
  'low': baSum.iloc[:, 2].tolist(),
  'high': baSum.iloc[:, 3].tolist(),
})

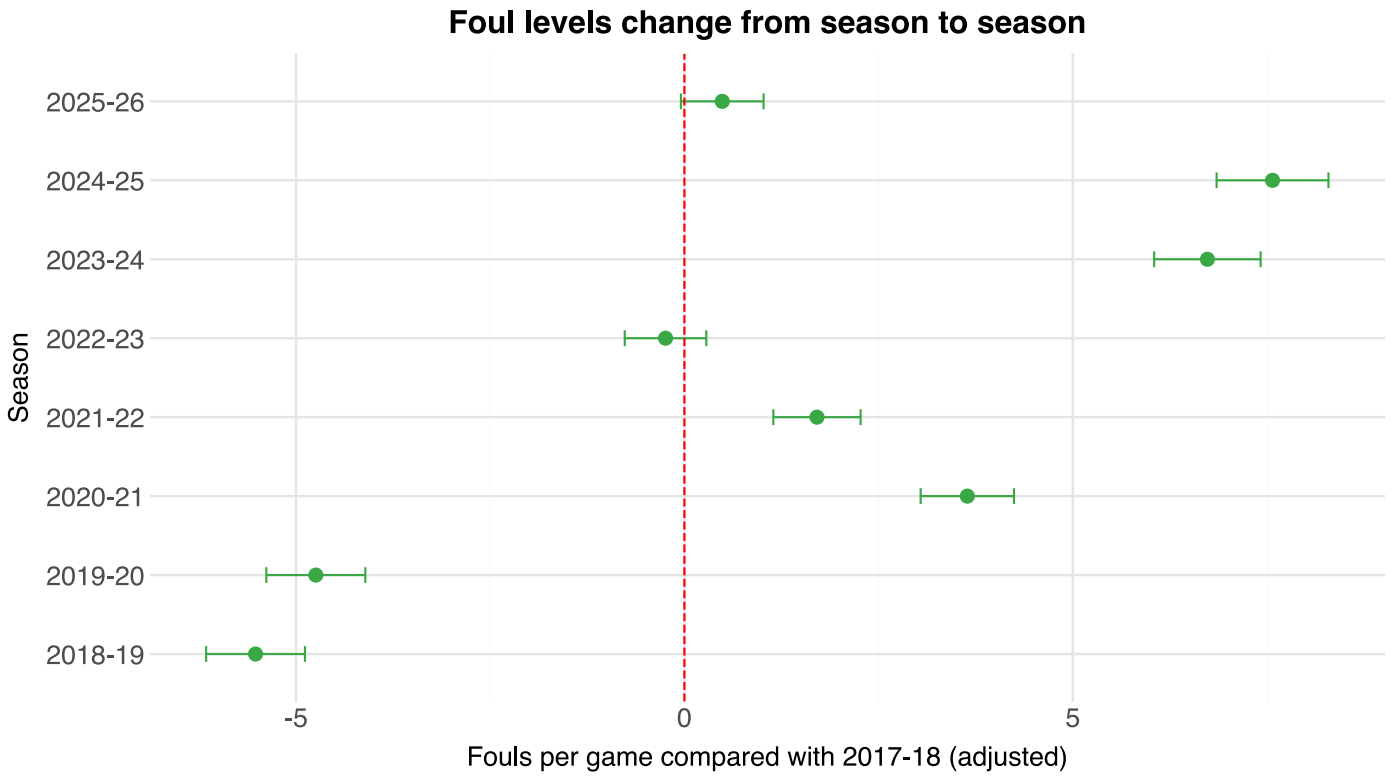
bothDf = (pl.concat([
    olsDf.with_columns(method = pl.lit('OLS (confidence interval)')),
    bayesDf.with_columns(method = pl.lit('Bayesian (credible interval)')),
  ])
  ## crew_home_win_pct has a very wide interval (0-1 scale), so it is left off the chart
  .filter(pl.col('term').is_in([k for k in niceNames if k != 'crew_home_win_pct']))
  .with_columns(label = pl.col('term').replace(niceNames))
)

(ggplot(bothDf, aes(x = 'coef', y = 'label'))
  + geom_point(size = 2.5, color = nbaGreen)
  + geom_errorbarh(aes(xmin = 'low', xmax = 'high'), height = 0.2, color = nbaGreen)
  + geom_vline(xintercept = 0, color = 'red', linetype = '--')
  + facet_wrap('~method')
  + scale_y_discrete(limits = [l for l in plotOrder if l != 'Crew: home-team win %'])
  + labs(x = 'Change in total fouls (95% interval)', y = '',
         title = 'Crews matter more than teams: two methods, same answer')
  + nbaTheme
)
```




```
seasonDf = (olsDf
  .filter(pl.col('term').str.starts_with('season_'))
  .with_columns(label = pl.col('term').str.slice(7, 4) + '-' + pl.col('term').str.slice(14, 2))
)

(ggplot(seasonDf, aes(x = 'coef', y = 'label'))
  + geom_point(size = 2.5, color = nbaGreen)
  + geom_errorbarh(aes(xmin = 'low', xmax = 'high'), height = 0.2, color = nbaGreen)
  + geom_vline(xintercept = 0, color = 'red', linetype = '--')
  + labs(x = 'Fouls per game compared with 2017-18 (adjusted)', y = 'Season',
         title = 'Foul levels change from season to season')
  + nbaTheme
)
```



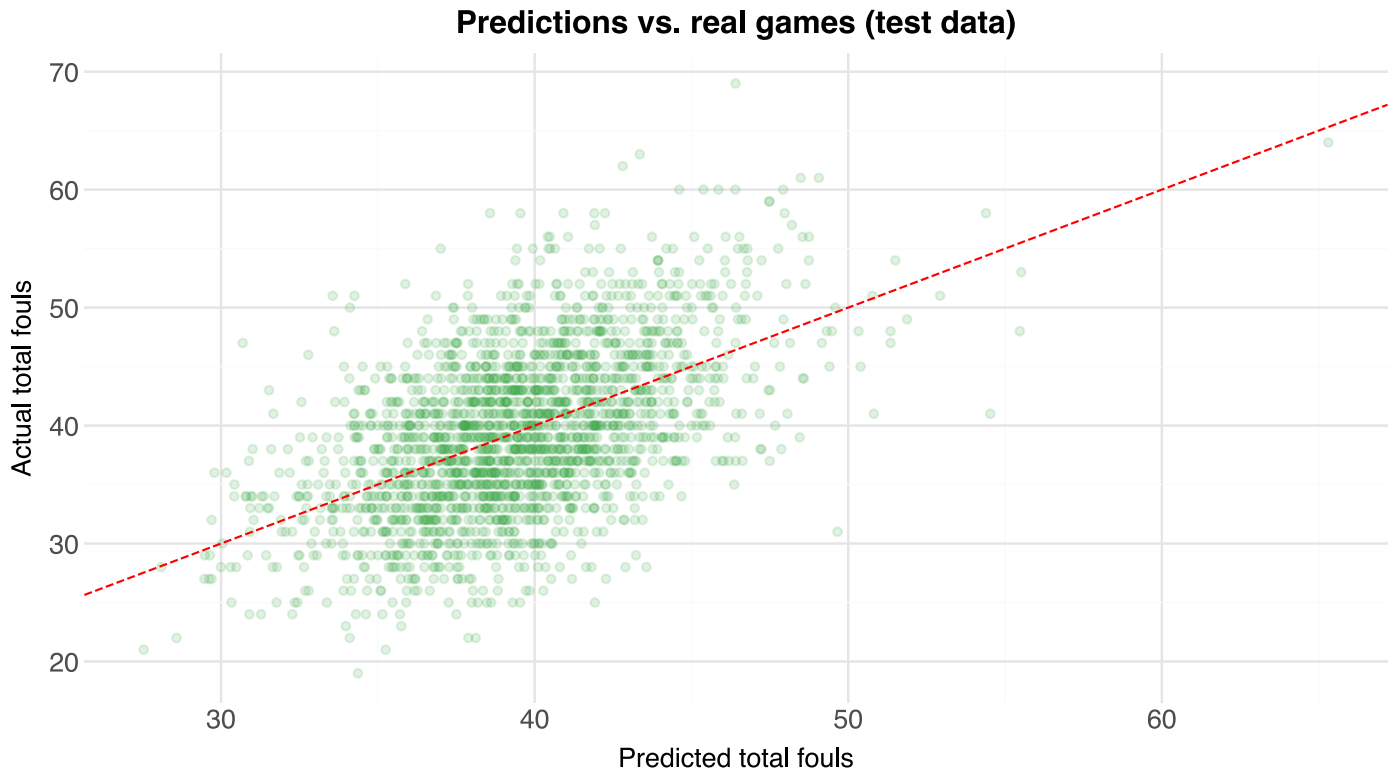
5.7 Predictive Fit on Test Data

```
y_pred = fit_05.predict(X_test)
print('Test RMSE:', round(root_mean_squared_error(y_test, y_pred), 2))

testPlot = pl.DataFrame({'actual': y_test.to_numpy(), 'predicted': np.asarray(y_pred)})

(ggplot(testPlot, aes(x = 'predicted', y = 'actual'))
  + geom_point(alpha = 0.15, color = nbaGreen)
  + geom_abline(slope = 1, intercept = 0, color = 'red', linetype = '--')
  + labs(x = 'Predicted total fouls', y = 'Actual total fouls',
         title = 'Predictions vs. real games (test data)')
  + nbaTheme
)
```

```
Test RMSE: 5.85
```



Points scatter evenly around the red line (perfect prediction), with no systematic over- or under-prediction. Most test games fall within about 6 fouls of the prediction.

5.8 Decision-Theoretic Fit: Expected Quadratic Loss

Our loss function is quadratic loss, so the expected loss of a forecasting approach is its average squared error on games it has not seen (test MSE). We compare the full model to two simple rules a coaching staff could use without a model.

```
trainSeason = gamesClean['season'].to_numpy()[X_train.index]
testSeason = gamesClean['season'].to_numpy()[X_test.index]
seasonMeans = pd.Series(y_train.to_numpy()).groupby(trainSeason).mean()

lossTable = pl.DataFrame({
    'forecast_rule': ['League average (all seasons)', 'Season average', 'Full model (C)'],
    'expected_loss_MSE': [
        mean_squared_error(y_test, np.full(len(y_test), y_train.mean())),
        mean_squared_error(y_test, seasonMeans.loc[testSeason].to_numpy()),
        mean_squared_error(y_test, y_pred),
    ],
}).with_columns(
    pl.col('expected_loss_MSE').round(2),
    typical_miss_fouls = pl.col('expected_loss_MSE').sqrt().round(2),
)
lossTable
```

forecast_rule	expected_loss_MSE	typical_miss_fouls
str	f64	f64
"League average (all seasons)"	47.26	6.87
"Season average"	47.24	6.87
"Full model (C)"	34.17	5.85

The full model has the lowest expected quadratic loss. Planning for the season average (MSE about 47) is no better than planning for the league average, while the full model lowers expected loss to about 34, a reduction of roughly 28%. In practical terms, the typical miss drops from about 6.9 to about 5.8 fouls, and large planning misses become less common. This is the measure that matches our business objective most directly.

6 Recommendations

6.1 Translating Crew Differences Into Free-Throw Points

We predict total fouls for an average 2025–26 matchup under a low-foul crew (10th percentile of 2025–26 crews), a typical crew (median), and a high-foul crew (90th percentile), holding everything else at typical values.

```
latest = gamesClean.filter(pl.col('season') == '2025-2026')

crewLevels = {
    'Low-foul crew': latest['crew_fouls_per_game'].quantile(0.10),
    'Typical crew': latest['crew_fouls_per_game'].median(),
    'High-foul crew': latest['crew_fouls_per_game'].quantile(0.90),
}

X_scen = pd.DataFrame(0.0, index = range(3), columns = X_train.columns)
X_scen['const'] = 1
X_scen['crew_fouls_per_game'] = list(crewLevels.values())
X_scen['crew_experience'] = latest['crew_experience'].median()
X_scen['crew_foul_diff'] = latest['crew_foul_diff'].median()
X_scen['crew_home_win_pct'] = latest['crew_home_win_pct'].median()
X_scen['home_team_fouls'] = latest['home_team_fouls'].median()
X_scen['away_team_fouls'] = latest['away_team_fouls'].median()
X_scen['season_2025_2026'] = 1

scenTable = fit_05.get_prediction(X_scen).summary_frame(alpha = 0.05)
scenTable.index = list(crewLevels.keys())

## ASSUMPTION: ~1.1 free throws per foul x ~78% made = ~0.85 points per foul
pointsPerFoul = 0.85
extraFouls = scenTable.loc['High-foul crew', 'mean'] - scenTable.loc['Low-foul crew', 'mean']

print(scenTable[['mean', 'mean_ci_lower', 'mean_ci_upper', 'obs_ci_lower', 'obs_ci_upper']].round(1))
print(f'\nHigh-foul vs. low-foul crew: {extraFouls:.1f} more fouls,')
print(f'about {extraFouls * pointsPerFoul:.1f} extra free-throw points in the game.')
```

```
               mean  mean_ci_lower  mean_ci_upper  obs_ci_lower  obs_ci_upper
Low-foul crew  37.5           37.1           37.9          25.9          49.1
Typical crew   39.1           38.8           39.5          27.5          50.7
High-foul crew 41.3           40.9           41.7          29.7          52.9

High-foul vs. low-foul crew: 3.8 more fouls,
about 3.2 extra free-throw points in the game.
```

A high-foul crew adds about **4 fouls per game** compared with a low-foul crew, or about **3 to 3.5 free-throw points**. In a league where many games are decided by a few points, this is a meaningful edge to plan for. The 95% prediction intervals for any single game are wide (about ±11 fouls), so the crew shifts the *expected* game, but cannot guarantee the outcome of one game.

6.2 Case Study: Lakers vs. Warriors, October 13, 2026

To test the model in a real pre-game setting, we predicted a game before it was played, using only information available at 9 AM ET on game day: the assigned crew (Mark Lindsay, Dedric Taylor, Nate Cearley) and each referee's and team's **2025–26** averages. This avoids the leakage issue described earlier, because all inputs come from the previous season.

```
crewToday = [nameFix.get(r, r) for r in ['Mark Lindsay', 'Dedric Taylor', 'Nate Cearley']]
homeTeam, awayTeam = 'LAL', 'GSW'
```

```
lastRefs = refStats.filter(pl.col('season') == '2025-2026')
statCols = ['fouls_per_game', 'experience', 'foul_diff', 'home_win_pct']
medianRef = {k: float(v) for k, v in lastRefs.select(statCols).median().row(0, named = True).items()}

crewList = []
for r in crewToday:
    row = lastRefs.filter(pl.col('referee') == r)
    if row.height == 0:
        print(f'{r} not found in 2025-2026 stats, using a typical referee instead.')
        crewList.append(medianRef)
    else:
        crewList.append({k: float(v) for k, v in row.select(statCols).row(0, named = True).items()})
crewNow = pl.DataFrame(crewList)

def teamFouls(abbr):
    return latest.filter(pl.col('home_team_abbr') == abbr)['home_team_fouls'][0]

rowNames = ['Actual crew (Oct 13)', 'Low-foul crew', 'Typical crew', 'High-foul crew']
X_new = pd.DataFrame(0.0, index = range(4), columns = X_train.columns)
X_new['const'] = 1
X_new['crew_fouls_per_game'] = [crewNow['fouls_per_game'].mean()] + list(crewLevels.values())
X_new['crew_experience'] = [crewNow['experience'].mean()] + [latest['crew_experience'].median()] * 3
X_new['crew_foul_diff'] = [crewNow['foul_diff'].mean()] + [latest['crew_foul_diff'].median()] * 3
X_new['crew_home_win_pct'] = [crewNow['home_win_pct'].mean()] + [latest['crew_home_win_pct'].median()] * 3
X_new['home_team_fouls'] = teamFouls(homeTeam)
X_new['away_team_fouls'] = teamFouls(awayTeam)
X_new['season_2025_2026'] = 1   ## closest season in the data

predTable = fit_05.get_prediction(X_new).summary_frame(alpha = 0.05)
predTable['crew'] = rowNames

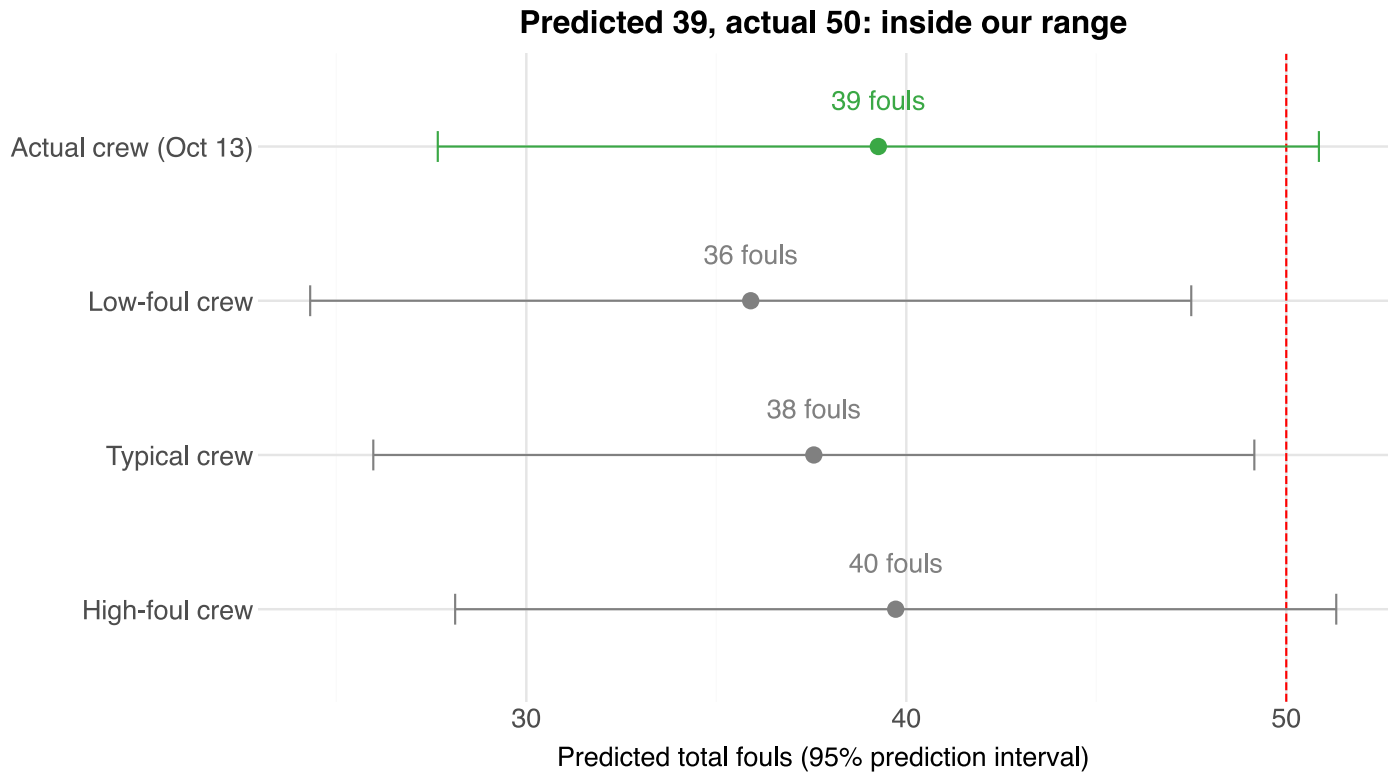
actualFouls = 50   ## from the official box score (both teams' personal fouls)

p = predTable.iloc[0]
print(f"Predicted: {p['mean']:.0f} fouls (95% prediction interval {p['obs_ci_lower']:.0f} to {p['obs_ci_upper']:.0f})")
print(f'Actual: {actualFouls} fouls | Inside interval: {p.obs_ci_lower <= actualFouls <= p.obs_ci_upper}')
```

```
Predicted: 39 fouls (95% prediction interval 28 to 51)
Actual: 50 fouls | Inside interval: True
```

```
predPlot = (pl.from_pandas(predTable[['crew', 'mean', 'obs_ci_lower', 'obs_ci_upper']])
  .with_columns(real = pl.col('crew') == 'Actual crew (Oct 13)')
)

(ggplot(predPlot, aes(x = 'mean', y = 'crew', color = 'real'))
  + geom_vline(xintercept = actualFouls, color = 'red', linetype = '--')
  + geom_errorbarh(aes(xmin = 'obs_ci_lower', xmax = 'obs_ci_upper'), height = 0.2)
  + geom_point(size = 3)
  + geom_text(aes(label = 'mean'), format_string = '{:.0f} fouls', nudge_y = 0.3)
  + scale_color_manual(values = {True: nbaGreen, False: 'grey'}, guide = None)
  + scale_y_discrete(limits = rowNames[::-1])
  + labs(x = 'Predicted total fouls (95% prediction interval)', y = '',
         title = 'Predicted 39, actual 50: inside our range')
  + nbaTheme
)
```



The model predicted 39 fouls with a 95% prediction interval of about 28 to 51. The actual game had 50 fouls, inside the interval but well above the point prediction. This is one game, and it was a **preseason** game, which is outside the regular-season data the model was trained on (preseason rotations and officiating can differ). It illustrates both how the model would be used and why the prediction interval, not just the point estimate, should guide planning.

6.3 Recommendations

1. **Scout the referees every game day.** Check the crew assignment at 9 AM ET and compute the crew's foul rate from the previous season (or season to date).
2. **High-foul crew expected:** plan for about 4 more fouls than with a low-foul crew. Manage minutes for foul-prone players, keep fouls-to-give in mind late in quarters, and play safer, less physical defense.
3. **Low-foul crew expected:** a more physical defensive approach carries less foul risk.
4. **Ignore crew experience and home/road bias** for planning total fouls; neither had a measurable effect.
5. **Use ranges, not single numbers.** Any single game can land roughly ±11 fouls from the prediction, so the model should shift planning, not replace in-game judgment.

6.4 Limitations and Next Steps

- **Pre-game inputs only.** Refit the model using prior-season or season-to-date referee and team averages so the in-sample estimates match the real pre-game use and remove the leakage from full-season averages.
- **More detailed data:** player fouls and minutes, foul types (shooting, offensive, loose-ball), team strategy data, and preseason/playoff games.
- **Model upgrades:** a Poisson regression for count outcomes, or a model with individual referee effects instead of a crew average.
- **Association, not causation.** The model shows that crews with higher foul rates are associated with more fouls; it does not prove that assigning a different crew would change a specific game by that amount.